



SonicSentinel AI



TECHNICAL DOCUMENTATION · V2.0

Developer Guide

Architecture · Code · Models · Operations

AI-powered real-time sound-event detection with two independently trained models – a Python XGBoost classifier and a Google Teachable Machine network – compared, explained and governed by configurable alert rules.

Flask 3 · SQLite

XGBoost · 299 features

Teachable Machine · TF.js

Explainable AI (TreeSHAP)

70 automated tests

FRONT MATTER

Document control

Table 1 Document information

Field	Value
Document	SonicSentinel AI – Developer Guide
Version	2.0 (this edition)
Status	Released for evaluation
Classification	Team and evaluators – competition prototype
System version	Python model <code>py-xgb-20260924-0800</code> · GTM export <code>gtm-TMv2-2026-09-24T20:51:32</code> · 70 automated tests
Source of truth	project repository <code>SonicSentinel AI/</code> and the generated <code>reports/</code>
Owner	Testing & documentation lead (see Module ownership)
Reviewed by	_____ Date: _____
Approved by	_____ Date: _____

Table 2 Revision history

Version	Date	Author	Changes
1.0	24 Sep 2026	SonicSentinel AI team	first developer guide: architecture, code, models, API, tests, deployment
2.0	24 Sep 2026	SonicSentinel AI team	new cover and chapter design; at-a-glance page; module ownership; quick start; sequence, decision-flow and state diagrams; real-data charts; annotated event page; colour-coded callouts; highlighted code; clickable cross-references; lists of figures and tables; glossary and index

TEAM

Module ownership

Each module has one owner who can explain it in the viva (SRS 1.8) and reviews changes to it. Replace the placeholders with the team members' names and GitHub usernames – the same assignment is recorded in `documentation/TEAM_CONTRIBUTIONS.md`.

Table 3 Module owners

Owner	Role	Modules	Functions to be able to explain
Member 1 _name / GitHub_	Team lead · integration	<code>src/services/analysis.py</code> , <code>decision.py</code> , <code>rules.py</code> , <code>run.py</code> , <code>src/__init__.py</code>	<code>analyze_upload</code> , <code>finalize</code> , <code>decide</code> , <code>evaluate_rule</code> , <code>combine</code> , look-alike check
Member 2 _name / GitHub_	Dataset & training	<code>scripts/</code> , <code>augmentation/</code> , <code>python_models/</code>	<code>build_dataset.py</code> , <code>random_augment</code> , <code>train_models.main</code> , <code>aggregate_scores</code>
Member 3 _name / GitHub_	GTM & front end	<code>gtm_model/</code> , <code>static/js/gtm.js</code> , <code>live.js</code> , <code>templates/</code>	GTM spectrogram, <code>classify</code> , live window loop, <code>prepare_gtm_samples --zip</code>
Member 4 _name / GitHub_	Audio pipeline	<code>audio_preprocessing/</code> , <code>feature_extraction/</code>	<code>validate_file</code> , <code>analyze_quality</code> , <code>preprocess_signal</code> , <code>segment</code> , <code>extract_features</code> , <code>similarity</code>
Member 5 _name / GitHub_	Testing & documentation	<code>tests/</code> , <code>documentation/</code> , README, report, blog, video	<code>conftest.py</code> isolation, benchmark and hidden-test scripts, this guide



NOTE Fill in before submission

Names in this table, in `AI_USAGE.md` ("Student modifications", "Verified by") and in `TEAM_CONTRIBUTIONS.md` must match the GitHub accounts that made the commits.

SUMMARY

At a glance

Key numbers of the current release, all taken from [reports/](#) and the test suite.

86.8 % Combined accuracy Python 0.7 + GTM 0.3 · 695 unseen clips	86.5 % Python accuracy macro-F1 0.863 · XGBoost	60.6 % GTM accuracy Teachable Machine run 3	1.00 Help-request recall critical recall ≥ 0.83 for all 5
1.47 s 30-s clip analysed target ≤ 8 s · max 1.74 s	0.11 s Live window target ≤ 3 s	20,020 Events benchmarked 5 concurrent users · 0 errors	0 Crashes 2,926 hidden-test runs
10 Sound classes + Unknown outcome	299 Acoustic features per 2-s segment	70 Automated tests 5 files · all passing	11 Database tables SQLite via SQLAlchemy

Table 4 The system in five sentences

Area	In one sentence
Input	uploads (WAV, MP3, FLAC, OGG, M4A), batch uploads and 2-second live microphone windows
Models	server-side XGBoost on 299 features and a browser-side Teachable Machine network, trained on the same TRAIN split and never sharing results
Decision	weighted combination, Unknown / overlap / look-alike checks, per-class alert rules with repeat confirmation, human review for doubtful cases
Explainability	full decision trace plus TreeSHAP feature-group contributions and the time span that triggered the class
Operations	role-based UI, audit trail, anomaly notifications, retention, /healthz , Docker and Render deployment

GET STARTED

Quick start – running in five minutes

- 1 **Install** Python 3.10 – 3.12, FFmpeg and Chrome / Edge.
- 2 **Create the environment** and install the libraries.
- 3 **Initialise** the database and default accounts.
- 4 **Start** the server and log in as `admin` / `Admin@12345`.
- 5 **Check** `/healthz` and run the tests.

```

Quick start – PowerShell POWERSHELL / SHELL
1 cd "SonicSentinel AI"
2 python -m venv .venv
3 .venv\Scripts\activate
4 pip install -r requirements.txt
5 python -m database.init_db
6 python run.py # open http://127.0.0.1:5000
7 # second terminal:
8 python -m pytest -q # expect: 70 passed

```

Table 5 Smoke test after installation

Check	Expected result
<code>http://127.0.0.1:5000/healthz</code>	<code>{"status": "ok", "checks": {"database": "ok", "python_model": "py-rgb-...", "gtm_model": "gtm-TMv2-..."}}</code>
Dashboard pills	Python version and "GTM · 10 classes" shown in green
Upload a clip from <code>audio_dataset/processed/test/Gunshot/</code>	event page shows both models, a decision trace and a Critical alert
Live → Start monitoring	microphone status "Active" and the red privacy banner



TIP Recommended settings for the demo

Admin → Settings: `python_weight` 0.7, `unknown_threshold` 0.35, `min_confidence` 0.60 – the combination that gave 86.8 % on the unseen test split.

Contents

Document control.....	1
Module ownership.....	2
At a glance.....	3
Quick start – running in five minutes.....	4
List of figures.....	7
List of tables.....	8
1. Introduction.....	1
1.1 Purpose of this guide.....	1
1.2 System summary.....	1
1.3 Technology stack.....	1
1.4 Conventions in this guide.....	2
2. Architecture.....	3
2.1 Component overview.....	3
2.2 Processing pipeline.....	4
2.3 Upload sequence.....	4
2.4 Live sequence.....	5
3. Repository structure.....	6
4. Development environment.....	7
4.1 Prerequisites.....	7
4.2 First-time setup.....	7
4.3 Environment variables.....	7
4.4 Useful development commands.....	8
4.5 Resetting local data.....	8
5. Configuration.....	9
5.1 Static configuration (config/settings.py).....	9
5.2 Runtime settings (database, Admin → Settings).....	9
5.3 Alert-rule schema (alert_rules/alert_rules.json).....	10
6. Audio processing library.....	11
6.1 Loading (loader.py).....	11
6.2 Validation (validation.py).....	11
6.3 Quality analysis (quality.py).....	11
6.4 Pre-processing and segmentation (preprocess.py).....	12
6.5 Augmentation (augmentation/augment.py).....	12
7. Feature extraction.....	14
7.1 The 299-dimensional feature vector.....	14
7.2 Audio fingerprint (fingerprint.py).....	14
7.3 Visuals (visuals.py).....	15
8. Python model.....	16
8.1 Training (python_models/train_models.py).....	16
8.2 Results of the current model (py-xgb-20260924-0800).....	17
8.3 Model bundle and inference (python_models/inference.py).....	19
8.4 Clip-level aggregation.....	19
9. Google Teachable Machine integration.....	20
9.1 Model format.....	20
9.2 Spectrogram computation (static/js/gtm.js).....	20
9.3 Training data for GTM (gtm_model/prepare_gtm_samples.py).....	20

9.4 GTM results.....	21
10. Decision engine.....	22
10.1 Inputs.....	22
10.2 Algorithm.....	22
10.3 Output.....	23
10.4 Example trace.....	24
11. Data model.....	25
11.1 Entity relationships.....	25
11.2 Event life cycle.....	26
12. Web application.....	27
12.1 Application factory (src/__init__.py).....	27
12.2 Security (src/security.py).....	27
12.3 Page routes.....	28
12.4 Front end.....	28
12.5 Anatomy of the event page.....	29
13. JSON API reference.....	32
14. Supporting services.....	34
15. Testing.....	35
15.1 Test suite.....	35
15.2 Writing a new test.....	36
15.3 Evaluation and benchmark scripts.....	36
16. Dataset and data scripts.....	38
16.1 Build pipeline.....	38
16.2 Current state of the data.....	38
17. Deployment.....	40
17.1 Windows (demo laptop).....	40
17.2 Docker.....	40
17.3 Render.....	40
17.4 Production checklist.....	40
18. Extending the system.....	41
18.1 Add a new sound class.....	41
18.2 Common changes.....	41
19. Known limitations.....	42
20. Developer troubleshooting.....	43
21. Glossary.....	44
Appendix A. Command reference.....	45
Appendix B. Related documents.....	46
Appendix C. Index.....	47

List of figures

Figure 1. Component architecture.....	3
Figure 2. Processing pipeline for one recording.....	4
Figure 3. Upload sequence – from the uploaded file to the final decision.....	5
Figure 4. Python model – precision, recall and F1 per class (TEST split).....	17
Figure 5. Confusion matrix of the Python model on 695 unseen TEST clips.....	18
Figure 6. Accuracy and macro-F1 as white noise gets stronger.....	18
Figure 7. Python, GTM and combined decision against the 85 % SRS target.....	21
Figure 8. Decision flow inside decide().....	23
Figure 9. Entity-relationship overview.....	25
Figure 10. Event status life cycle (EVENT_STATUSES).....	26
Figure 11. Event page (blocks 1–3) – a correctly detected Gunshot: Acceptable Match, Critical alert, TreeSHAP explanation.....	29
Figure 12. Event page (blocks 4–6) – waveform, spectrogram, audio quality and metadata.....	30
Figure 13. Event page (blocks 7–10) – model comparison, segments, alerts and reviews.....	30
Figure 14. Hidden-test readiness per degradation (reports/hidden_test_robustness.csv).....	37
Figure 15. Similar-event pairs – event recall vs. look-alike false alarms.....	37
Figure 16. Clips per class in the current dataset build.....	39

List of tables

Table 1. Document information.....	1
Table 2. Revision history.....	1
Table 3. Module owners.....	2
Table 4. The system in five sentences.....	3
Table 5. Smoke test after installation.....	4
Table 6. System summary.....	1
Table 7. Technology stack.....	1
Table 8. Component overview.....	3
Table 9. Repository structure.....	6
Table 10. Environment variables.....	7
Table 11. Static configuration (config/settings.py).....	9
Table 12. Runtime settings (database, Admin → Settings).....	10
Table 13. Validation (validation.py).....	11
Table 14. Quality analysis (quality.py).....	12
Table 15. Augmentation (augmentation/augment.py).....	12
Table 16. The 299-dimensional feature vector.....	14
Table 17. Training (python_models/train_models.py).....	16
Table 18. Results of the current model (py-xgb-20260924-0800).....	17
Table 19. Results of the current model (py-xgb-20260924-0800) – Class.....	17
Table 20. Results of the current model (py-xgb-20260924-0800) – Noise condition (reports/noise_robustness.csv).....	18
Table 21. GTM results.....	21
Table 22. Entity relationships.....	25
Table 23. Security (src/security.py).....	27
Table 24. Page routes.....	28
Table 25. Event page blocks and the code behind them.....	30
Table 26. JSON API reference.....	32
Table 27. Supporting services.....	34
Table 28. Test suite.....	35
Table 29. Evaluation and benchmark scripts.....	36
Table 30. Current state of the data.....	38
Table 31. Current state of the data – Class (current build).....	39
Table 32. Common changes.....	41
Table 33. Known limitations.....	42
Table 34. Developer troubleshooting.....	43
Table 35. Glossary.....	44
Table 36. Appendix B. Related documents.....	46



CHAPTER 01

Introduction

What this guide covers, the system in one page and the technology stack.

SRS SRS 1.1 · 1.9 Interface requirements

1.1 Purpose of this guide

This guide is for developers who build, maintain, test, deploy or extend SonicSentinel AI. It describes the real code in the repository: module layout, data flow, algorithms, database schema, API, configuration, tests and deployment. Every number in this document comes from the code or from the generated reports in `reports/` (model `py-xgb-20260924-0800`, GTM export of 2026-09-24 20:51 UTC).

For how to use the application, see the separate **SonicSentinel AI User Guide**.

1.2 System summary

SonicSentinel AI is a Flask web application that classifies audio into ten sound classes with **two independently trained models** – a Python XGBoost model on the server and a Google Teachable Machine (GTM) TensorFlow.js model in the browser – then compares them, applies configurable alert rules and routes doubtful results to a human reviewer.

Table 6 System summary

Property	Value
Classes (<code>config/settings.py</code> → <code>CLASSES</code>)	Machinery Fault, Glass Breaking, Alarm or Siren, Vehicle Horn, Animal Sound, Gunshot, Panic Scream, Aggression, Person Asking for Help, Background Noise (+ Unknown as a decision outcome)
Input	upload (WAV, MP3, FLAC, OGG, M4A, ≤ 25 MB, 0.5 s – 300 s), batch upload, live microphone (2-s windows)
Python model	XGBoost in a scikit-learn Pipeline (StandardScaler → XGBClassifier), 299 features per 2-s segment
GTM model	Teachable Machine audio project, TMv2 / speech-commands 0.4.0 BROWSER_FFT, 43 × 232 spectrogram input
Storage	SQLite via Flask-SQLAlchemy (11 tables); files under <code>data/</code>
Code size	≈ 6,500 lines of Python in 60 files, plus Jinja templates and 6 JavaScript files (app, gtm, live, batch, lab, wav)
Tests	70 pytest tests in 5 files

1.3 Technology stack

Table 7 Technology stack

Layer	Technology (from <code>requirements.txt</code>)
Web framework	Flask ≥ 3.0, Flask-SQLAlchemy ≥ 3.1, Flask-Login ≥ 0.6.3

Layer	Technology (from <code>requirements.txt</code>)
WSGI server	waitress ≥ 3.0 (Windows), gunicorn ≥ 21.2 (Linux / Docker)
Audio	librosa ≥ 0.10.2, soundfile ≥ 0.12, soxr ≥ 0.3, noisereduce ≥ 3.0, SciPy ≥ 1.11, FFmpeg (MP3 / M4A decoding)
Machine learning	NumPy ≥ 1.26, pandas ≥ 2.1, scikit-learn ≥ 1.4, XGBoost ≥ 2.0, joblib ≥ 1.3
Reports	Matplotlib ≥ 3.8, openpyxl ≥ 3.1
Front end	Jinja2 templates, Tailwind CSS (CDN, custom palette), Chart.js, TensorFlow.js 1.3.1 + @tensorflow-models/speech-commands 0.4.0 (the versions Teachable Machine exports for)
Testing	pytest ≥ 8.0
Optional	sounddevice (record_samples.py), pyttsx3 (generate_help_phrases_tts.py)

1.4 Conventions in this guide

- `monospace` = file, function, setting or command. Paths are relative to the project root `SonicSentinel AI/`.
- "Segment" = a fixed-length piece of audio classified by the models (2 s by default). "Window" = one live-microphone segment.
- Commands are shown for Windows PowerShell; on Linux / macOS replace `.venv\Scripts\activate` with `source .venv/bin/activate`.



TIP Reading on screen

Every chapter reference, every entry in the contents and every entry in the lists of figures and tables is a clickable link. In Word use Ctrl + click; in the PDF use the bookmarks panel.



CHAPTER 02

Architecture

How the browser, the Flask server, the two models and the database work together.

SRS SRS 1.2 Steps 2, 10–12 · 1.8 model independence

2.1 Component overview

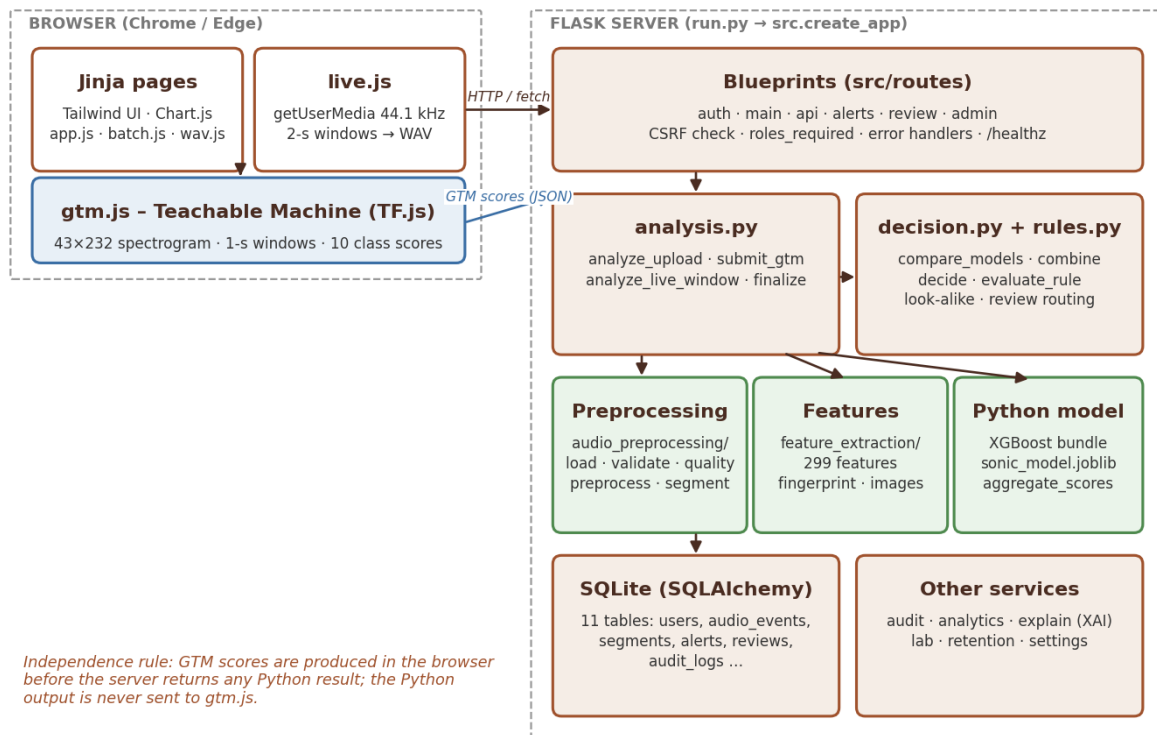


Figure 1 Component architecture

Table 8 Component overview

Component	Responsibility	Main files
Browser UI	pages, uploads, charts, toasts, live capture	templates/, static/js/app.js, batch.js, live.js, lab.js, wav.js (WAV encoder)
GTM client	computes the Teachable Machine spectrogram and runs the TF.js model on every segment	static/js/gtm.js, templates/partials/gtm_scripts.html
Routes	HTTP endpoints, access control, rendering	src/routes/*.py
Analysis service	orchestrates validation → quality → pre-processing → Python model → GTM hand-off → decision → storage	src/services/analysis.py
Decision engine	pure functions: model comparison, combined score, rule evaluation, look-alike check, review routing	src/services/decision.py, src/services/rules.py

Component	Responsibility	Main files
Audio / features / model	reusable libraries shared by the app, training and scripts	<code>audio_preprocessing/</code> , <code>feature_extraction/</code> , <code>python_models/</code> , <code>augmentation/</code>
Persistence	ORM models and seeding	<code>src/models.py</code> , <code>database/init_db.py</code>
Support services	audit and notifications, analytics, explainable AI, Robustness Lab, retention, runtime settings	<code>src/services/</code>

2.2 Processing pipeline

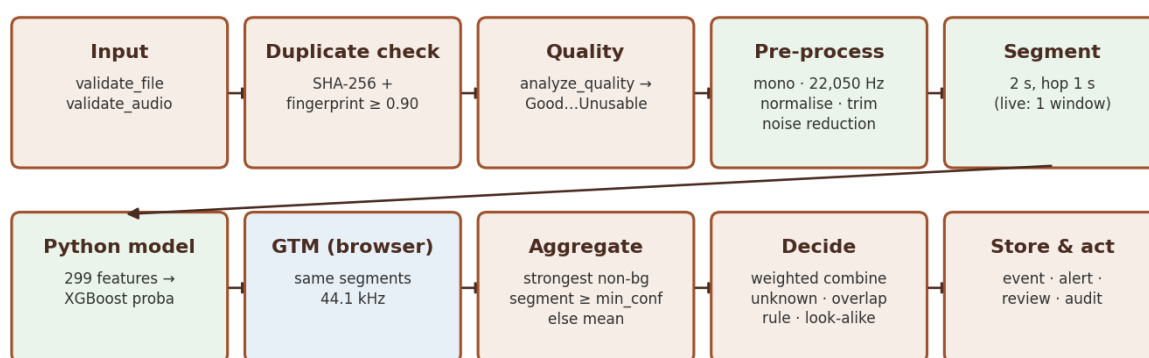


Figure 2 Processing pipeline for one recording

2.3 Upload sequence

- 1 `POST /upload` (form) or `POST /api/upload` (fetch) saves the file under `data/uploads/` and calls `analysis.analyze_upload()`.
- 2 `analyze_upload` validates, checks SHA-256 and fingerprint duplicates, analyses quality, pre-processes, segments, writes one 44.1 kHz WAV per segment to `data/segments/<audio_id>/seg_NNN.wav`, runs the Python model, draws the waveform and spectrogram, and commits the event with `gtm_status = "pending"`.
- 3 The event page loads `gtm.js`. For each segment it fetches the WAV (`/media/segment/<id>`), decodes it at 44.1 kHz, classifies it and posts `{"segments": [{class: score}, ...]}` to `POST /api/events/<audio_id>/gtm`.
- 4 `submit_gtm()` stores the per-segment GTM scores and calls `finalize()`, which aggregates both models, runs `decision.decide()` and creates an `Alert` if needed.
- 5 If no GTM model is installed, `finalize()` runs immediately after step 2 with Python only (status "Uncertain Result").

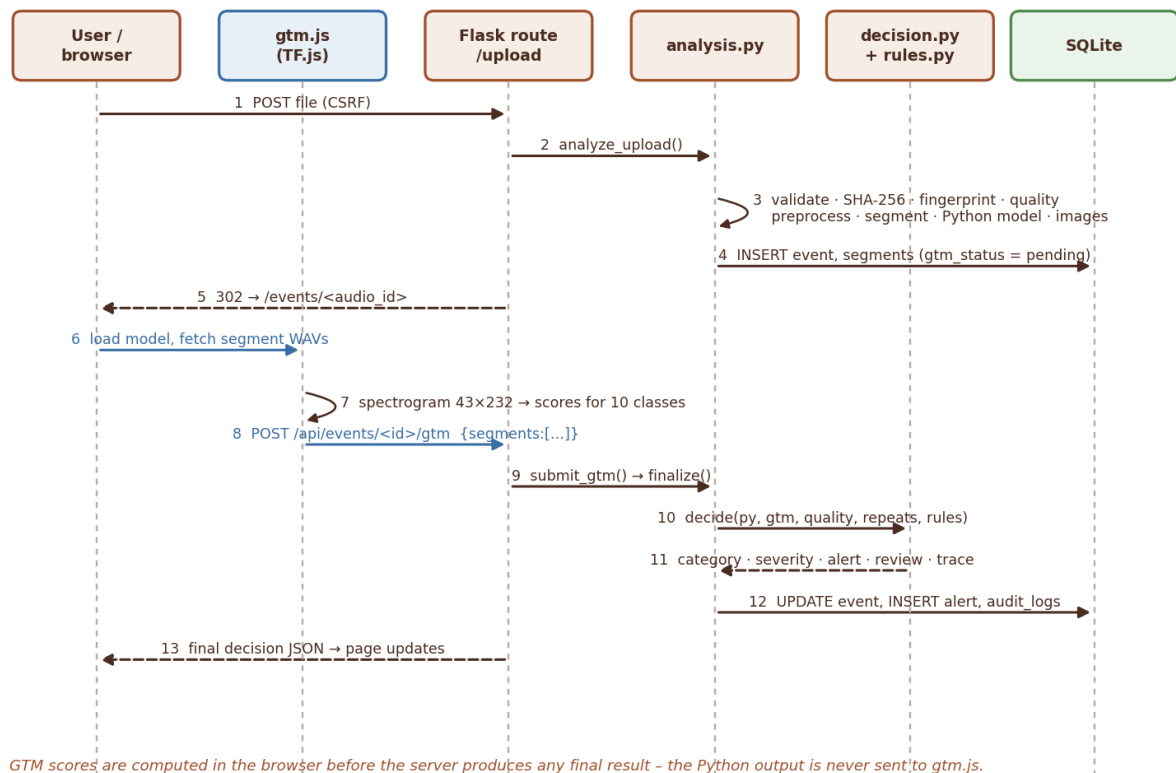


Figure 3 Upload sequence – from the uploaded file to the final decision

2.4 Live sequence

- 1 `live.js` calls `POST /api/live/start` → a `LiveSession` with code `LS-<timestamp>-<user>-<hex>` is created and audited (`mic_start`).
- 2 A `ScriptProcessorNode` on a 44.1 kHz `AudioContext` fills a buffer; every `live_window_seconds` (2 s) the window is cut. If the previous window is still being analysed the new one is dropped so the monitor never falls behind real time.
- 3 GTM classifies the window **in the browser first**, then the WAV and the GTM scores are posted to `POST /api/live/window`.
- 4 `analyze_live_window()` skips silent windows, stores a `LIV-...` event, runs the Python model and `finalize()`. Repeated detection counts consecutive windows of the same session inside `repeat_window_seconds`.
- 5 The JSON response (and any alert) is rendered in the live table; `POST /api/live/stop` closes the session (`mic_stop`).



SRS Model independence (SRS 1.8)

GTM always runs on the client and receives only audio. The Python result is produced on the server and is never passed to `gtm.js`. For uploads, the event page does not show a final decision until GTM has posted its own scores. For live windows the GTM scores are attached to the request before the server sees the audio.



CHAPTER 03

Repository structure

Where every part of the code lives.

SRS SRS 1.10 Deliverable 2 (source code)

Table 9 Repository structure

Path	Contents
<code>run.py</code>	entry point: <code>app = create_app(); python run.py [--host] [--port] [--debug]</code>
<code>config/settings.py</code>	paths, class list, audio constants, roles, statuses, default runtime settings, look-alike pairs, secret key
<code>audio_preprocessing/</code>	<code>loader.py</code> (soundfile + FFmpeg fallback), <code>validation.py</code> , <code>quality.py</code> , <code>preprocess.py</code>
<code>feature_extraction/</code>	<code>features.py</code> (299 features), <code>fingerprint.py</code> (near-duplicates), <code>visuals.py</code> (waveform / spectrogram PNG)
<code>augmentation/augment.py</code>	noise, shift, pitch, stretch, volume, reverb, distance, device; CLI for the training split
<code>python_models/</code>	<code>train_models.py</code> (tuning, comparison, reports), <code>inference.py</code> (model wrapper, aggregation), <code>saved/sonic_model.joblib</code>
<code>gtm_model/</code>	<code>prepare_gtm_samples.py</code> (WAV / playlist / TM zip export), <code>tm_upload_fix.js</code> , <code>model/</code> (model.json, weights.bin, metadata.json)
<code>alert_rules/</code> <code>alert_rules.json</code>	one rule per class + <code>_help</code> block
<code>database/init_db.py</code>	creates tables, seeds default accounts and model versions
<code>src/</code>	Flask app: <code>__init__.py</code> (factory, error handlers, /healthz), <code>models.py</code> , <code>security.py</code> , <code>extensions.py</code> , <code>routes/</code> , <code>services/</code>
<code>templates/</code>	Jinja pages; <code>partials/</code> (macros, icons, GTM scripts, auth art), <code>admin/</code> , <code>auth/</code> , <code>events/</code> , <code>errors/</code>
<code>static/</code>	<code>css/app.css</code> (theme on top of Tailwind), <code>js/</code> (app, gtm, live, batch, lab, wav), <code>img/</code> (original SVG illustrations)
<code>scripts/</code>	dataset import / build / TTS / recording, benchmark, hidden-test robustness, look-alike report
<code>tests/</code>	pytest suite with isolated fixtures
<code>data/</code>	runtime data: database, uploads, segments, images, lab files, dataset metadata (git-ignored except metadata)
<code>reports/</code>	generated model, performance and robustness reports
<code>documentation/</code>	architecture, dataset guide, data dictionary, testing, deployment, GTM log, project report, blog, user and developer guides



CHAPTER 04

Development environment

Set up, run, debug and reset a local development environment.

SRS SRS 1.10 Deliverables 9–10

4.1 Prerequisites

- Python 3.10 – 3.12 (the code uses `X | None` type hints, so 3.10 is the minimum).
- FFmpeg on PATH for MP3 / M4A (`winget install Gyan.FFmpeg`). WAV, FLAC and OGG work without it.
- Chrome or Edge for live monitoring and GTM (Web Audio + TF.js).

4.2 First-time setup

```
PowerShell – project root POWERSHELL / SHELL
1 cd "SonicSentinel AI"
2 python -m venv .env
3 .env\Scripts\activate
4 pip install -r requirements.txt
5 python -m database.init_db # tables + default accounts
6 python run.py --debug # http://127.0.0.1:5000
```

`setup_windows.bat` performs the same steps; `run_windows.bat` starts the app; `run_server.bat` runs waitress with automatic restart and logs to `logs/server.log`.

4.3 Environment variables

Table 10 Environment variables

Variable	Default	Purpose
<code>SONIC_SECRET_KEY</code>	"change-this-secret-key-in-production"	Flask session signing – always set in production
<code>SONIC_DATA_DIR</code>	<code>data/</code>	root for uploads, segments, images, database
<code>SONIC_DB_PATH</code>	<code>data/sonicsentinel.db</code>	SQLite file
<code>SONIC_MODEL_PATH</code>	<code>python_models/saved/sonic_model.joblib</code>	Python model bundle
<code>SONIC_GTM_DIR</code>	<code>gtm_model/model/</code>	folder with model.json, metadata.json, weights.bin
<code>SONIC_RULES_PATH</code>	<code>alert_rules/alert_rules.json</code>	alert-rule file
<code>HOST, PORT</code>	127.0.0.1, 5000	bind address for <code>run.py</code>

**SECURITY Secrets**

Never commit a real `SONIC_SECRET_KEY`. Anyone who knows it can forge session cookies and log in as any user. Set it as an environment variable on the server (Render generates one automatically).

The test suite uses these variables to run against a temporary folder, database, model and rule file (see [chapter 15](#)).

4.4 Useful development commands

Developer commands		POWERSHELL / SHELL
1	<code>python -m pytest -q</code>	<i># full test suite (70 tests)</i>
2	<code>python -m pytest tests/test_decision_rules.py -q</code>	<i># one file</i>
3	<code>python -m python_models.train_models --fast</code>	<i># quick training check</i>
4	<code>python scripts/benchmark_performance.py</code>	<i># NFR performance / scalability</i>
5	<code>python scripts/hidden_test_robustness.py</code>	<i># noise, echo, MP3 ... on TEST clips</i>
6	<code>python scripts/lookalike_report.py</code>	<i># similar-event pairs</i>
7	<code>python -m gtm_model.prepare_gtm_samples --max-per-class 400 --zip</code>	

4.5 Resetting local data

Stop the server, delete `data/sonicsentinel.db` (and optionally `data/uploads`, `data/segments`, `data/images`), then run `python -m database.init_db.create_app()` also calls `db.create_all()` and `seed_defaults()` on every start, so missing tables and accounts are recreated automatically.

**WARNING Deleting the database**

Removing `sonicsentinel.db` deletes every event, alert, review and audit record. Copy the file first if the data is needed as evidence (model-comparison report, screenshots).



CHAPTER 05

Configuration

Static constants, runtime settings and the alert-rule schema.

SRS FR xxxv–xxxvi · FR liii · FR lxxx

5.1 Static configuration (config/settings.py)

Table 11 Static configuration (config/settings.py)

Constant	Value	Used for
CLASSES	10 class names (order matters for the model)	labels of both models; must match GTM <code>metadata.json</code> spelling
BACKGROUND_CLASS, UNKNOWN_CLASS	"Background Noise", "Unknown"	aggregation and decision special cases
CRITICAL_RECALL_CLASSES	Gunshot, Glass Breaking, Panic Scream, Aggression, Person Asking for Help	model selection tie-break, reports
SUPPORTED_FORMATS	wav, mp3, flac, ogg, m4a	upload validation
TARGET_SR / GTM_SR	22,050 Hz / 44,100 Hz	Python pipeline / GTM input
MAX_UPLOAD_MB	25	per-file limit (request limit is 20 × this for batch)
MIN_DURATION_S / MAX_DURATION_S	0.5 / 300	duration validation
N_MFCC / N_MELS	40 / 64	feature extraction
SEVERITY_LEVELS	Informational, Low, Medium, High, Critical	rules, UI
QUALITY_LEVELS	Good, Acceptable, Poor, Unusable	quality analysis
EVENT_STATUSES	Uploaded, Classified, Uncertain, Alert Generated, Manual Review, Reviewed, Closed	event life cycle
ROLES	user, reviewer, security, maintenance, admin	access control
LOOKALIKE_PAIRS	class → [(look-alike label, real-world description)]	look-alike check (SRS Step 14)

5.2 Runtime settings (database, Admin → Settings)

`DEFAULT_RUNTIME_SETTINGS` holds defaults; administrators override them and the values are stored in the `settings` table. Always read them with `settings_service.get_settings()`, which merges stored values over the defaults. `update_settings()` casts form values to the default's type and ignores unknown keys and missing fields.

Table 12 Runtime settings (database, Admin → Settings)

Key	Default	Effect
<code>segment_seconds / segment_hop_seconds</code>	2.0 / 1.0	upload segmentation (model trained with 2 s – retrain if changed)
<code>live_window_seconds</code>	2.0	live window length
<code>min_confidence</code>	0.60	Medium confidence threshold, aggregation threshold, review trigger
<code>top2_margin</code>	0.15	uncertainty / review trigger
<code>acceptable_match_diff</code>	0.20	max $ \Delta\text{conf} $ for "Acceptable Match"
<code>unknown_threshold</code>	0.35	combined confidence below → Unknown
<code>overlap_threshold</code>	0.25	≥ 2 non-background classes above → overlap
<code>python_weight</code>	0.5 (0.7 recommended)	weight of the Python model in <code>combine()</code>
<code>lookalike_margin</code>	0.20	minimum gap between an event and its look-alike
<code>repeat_window_seconds</code>	10	live repeated-detection window
<code>noise_reduction</code>	True	spectral gating before the Python model
<code>background_noise_limit_db</code>	-20.0	noise-floor alert for Background Noise
<code>audio_retention_days / record_retention_days</code>	90 / 365	<code>retention.apply_retention()</code>

5.3 Alert-rule schema (alert_rules/alert_rules.json)

● ● ● alert_rules/alert_rules.json	JSON
<pre> 1 "Gunshot": { 2 "severity": "Critical", "generate_alert": true, "min_confidence": 0.70, "top2_margin": 0.15, 3 "required_consecutive": 2, "strong_confidence": 0.85, "require_model_agreement": false, 4 "min_quality": "Poor", "manual_review_if_unconfirmed": true, 5 "escalate_after_repeats": 0, "escalate_to": "", 6 "recommended_action": "CRITICAL: possible gunshot. Alert security immediately, ...", 7 "audience": ["security"] 8 }</pre>	

`rules.load_rules()` reads the file, `save_rules()` writes it (Admin → Alert rules), `validate_rule()` rejects invalid severities, qualities, probabilities outside 0 – 1 and `required_consecutive < 1`. A class without a rule gets `default_rule()` (Low, no alert, manual review if unconfirmed). Background Noise additionally supports `noise_alert_severity` and `noise_alert_action`.



CHAPTER 06

Audio processing library

Loading, validation, quality analysis, pre-processing and augmentation.

SRS Steps 3–4, 13 · FR viii, x–xv, xix, xxxvii

`audio_preprocessing/` is used **identically** by the web application, the training script, the evaluation scripts and the Robustness Lab, so the model always sees audio prepared the same way it was trained on.

6.1 Loading (`loader.py`)

- `load_audio(path)` first tries `soundfile` (WAV, FLAC, OGG); if that fails it pipes the file through FFmpeg (`ffmpeg -i ... -f wav -acodec pcm_f32le pipe:1`) for MP3 / M4A. Native sample rate and channel count are kept.
- Returns `AudioData(samples, sample_rate, channels, bit_depth, format, file_size)`; `.duration` and `.metadata(filename)` feed the event record.
- Raises `AudioDecodeError` for empty files, non-finite samples or undecodable data (missing FFmpeg is reported explicitly).
- `load_bytes(raw, suffix)` decodes in-memory audio (live windows) via a temporary file.

6.2 Validation (`validation.py`)

Table 13 Validation (`validation.py`)

Check	Rule	Result
Extension	in <code>SUPPORTED_FORMATS</code>	error
Size	> 0 bytes and ≤ 25 MB	error
Integrity	decodes without exception	error "File integrity check failed ..."
Sample rate	≥ 8,000 Hz (warning below 16 kHz)	error / warning
Channels	1 – 8	error
Duration	0.5 s – 300 s (skipped for live windows)	error
Signal	peak ≥ 1e-4	error "recording is silent"

`validate_file()` returns a `ValidationResult(ok, errors, warnings, audio)`; the decoded audio is reused so the file is only read once.

6.3 Quality analysis (`quality.py`)

`analyze_quality(samples, sr)` works on the **raw, un-normalised** signal and returns the label, a list of human-readable issues and the measurements (RMS, peak, silence ratio, clipping ratio, noise floor, SNR, dropouts, spectral flatness). The label starts at Good and can only get worse:

Table 14 Quality analysis (quality.py)

Condition	Measurement	Label
Encoding problem	non-finite samples	Unusable
Silent	peak < 1e-4 or > 97 % of frames below -60 dB	Unusable
Too short / too long	< 0.5 s / > 300 s	Unusable / Poor
Clipping	> 1 % samples at $ x \geq 0.999$ / > 0.1 %	Poor / Acceptable
Low level	RMS < -50 dBFS / < -38 dBFS	Poor / Acceptable
Background noise	SNR (95th - 10th percentile frame dB) < 6 dB and flatness > 0.30 / SNR < 12 dB	Poor or Acceptable / Acceptable
Missing frames	runs of exact zeros ≥ 20 ms inside the clip (1 - 2 / > 2)	Acceptable / Poor

Low SNR alone is not penalised because continuous sounds (sirens, machinery hum) naturally have a small dynamic range; the spectral-flatness test separates them from real broadband noise.

`noise_floor_dbfs` is stored as `noise_level_db` and drives the Background-Noise alert.

6.4 Pre-processing and segmentation (preprocess.py)

```

● ● ● audio_preprocessing/preprocess.py – API PYTHON
1 preprocess_signal(samples, sr, trim=True, denoise=True) -> (clean @ 22,050 Hz,
trim_offset_s)
2     to_mono -> resample (soxr_hq) -> peak_normalize(0.95)
3     -> trim_silence(top_db=40) -> reduce_noise (noisereduce, stationary,
prop_decrease=0.75)
4     -> peak_normalize
5
6 segment(y, sr, seg_seconds=2.0, hop_seconds=1.0) -> [(start_s, end_s, samples)]
7     clip shorter than one segment -> one zero-padded segment
8     trailing remainder < 50 % of a segment is dropped
9
10 prepare_for_gtm(samples, sr, 44100) # mono, 44.1 kHz, peak-normalised, NO noise
reduction

```

Segment timestamps are shifted by the trim offset so they refer to the **original** recording. Live windows are not trimmed and are padded / truncated to exactly `live_window_seconds`.

6.5 Augmentation (augmentation/augment.py)

Table 15 Augmentation (augmentation/augment.py)

Function	Parameters (random ranges)
<code>add_noise</code>	SNR 5 - 25 dB; white noise or a real Background Noise clip
<code>time_shift</code>	roll by ± 30 %
<code>pitch_shift</code>	± 2 semitones
<code>time_stretch</code>	rate 0.85 - 1.15

Function	Parameters (random ranges)
volume	-12 to +6 dB
reverb	RT60 0.15 – 0.6 s, exponentially decaying noise impulse response, 60 / 40 dry / wet
distance	5 – 30 m: 4th-order low-pass (9 kHz – 200 Hz·m, ≥ 1.5 kHz) and attenuation
device	cheap microphone: 300 Hz – 4 kHz band-pass + tanh saturation

`random_augment()` chains 1 – 3 random operations and renormalises to 0.9 peak. The CLI (`python -m augmentation.augment --per-clip 2`) augments **only the training split**; augmented files keep `parent_audio_id` so they stay in their parent's split and cross-validation group.



CHAPTER 07

Feature extraction

The 299-dimensional feature vector, audio fingerprints and visuals.

SRS Step 6 · FR xx-xxii, lxxiii-lxxiv

7.1 The 299-dimensional feature vector

`feature_extraction/features.py` → `extract_features(y, sr=22050)` turns one pre-processed segment into a fixed-length float32 vector (STFT: `n_fft` 2048, `hop` 512). `feature_names()` returns the names in the same order.

Table 16 The 299-dimensional feature vector

Block	Statistics	Dimensions
MFCC (40 coefficients from a 64-band log-Mel)	mean, std	80
Δ -MFCC	mean	40
Log-Mel spectrogram (64 bands)	mean, std	128
Chroma (12)	mean, std	24
Spectral contrast (7)	mean	7
ZCR, RMS, centroid, bandwidth, roll-off, onset strength	mean, std, max (centroid, bandwidth, roll-off in kHz)	18
Spectral flatness, tempo (/100)	mean / value	2
Total		299

NaN / $\pm\text{inf}$ are replaced by 0; digital silence gets a $1\text{e-}6$ offset. `FEATURE_VERSION = "v1"` is part of the training cache key – **bump it whenever the feature code changes** so cached `.npz` files under `data/features/<tag>/` are not reused.



WARNING Changing a feature

Any change to `extract_features()` changes the meaning of the vector. Bump `FEATURE_VERSION`, update `feature_names()` and retrain – an old model bundle with new features gives silently wrong predictions.

7.2 Audio fingerprint (fingerprint.py)

- `compute_fingerprint(y, sr)` at 11,025 Hz: a 32-band mean log-Mel profile (mean-removed, so volume-independent) and a 10 Hz log-RMS envelope ($\leq 3,000$ values).
- `similarity(a, b)`: cosine of the profiles; if ≥ 0.85 and the durations are within a factor of 3.3, the best normalised cross-correlation of the envelopes is added: `0.3 · spectral + 0.7 · envelope`.

- `analyze_upload` compares a new upload with the last 2,000 non-live events; ≥ 0.90 marks a near-duplicate (re-encoded, trimmed or volume-changed copy) and notifies the administrator. Identical files are caught earlier by SHA-256.

7.3 Visuals (visuals.py)

`save_waveform()` draws the waveform with segment boundaries and the deciding segment highlighted; `save_spectrogram()` draws a 128-band Mel spectrogram. PNGs are written to `data/images/` and served through `/media/event/<audio_id>/<kind>`.



CHAPTER 08

Python model

Training, tuning, selection, results and inference of the Python model.

SRS Steps 7–8 · FR xxiii–xxvi · NFR 4

8.1 Training (python_models/train_models.py)

```
python_models/train_models.py – CLI POWERSHELL / SHELL

1 python -m python_models.train_models # svm, rf, xgb, mlp – full
  grids
2 python -m python_models.train_models --models xgb,mlp,rf # what was used for the
  current model
3 python -m python_models.train_models --fast # small grids
```

- 1 Read `data/dataset_metadata.csv`: training rows (including augmented copies), validation and test rows (originals only).
- 2 Extract segment features with an on-disk cache keyed by `FEATURE_VERSION`, segment length, hop, noise reduction and sample rate.
- 3 Drop quiet segments of event clips from training (energy < 25 % of the clip maximum) – a silent segment of a gunshot clip is not a gunshot.
- 4 For every candidate: `Pipeline([StandardScaler, classifier])`, `GridSearchCV(scoring="f1_macro")` with `GroupKFold(3)` grouped by the original Audio ID, so segments and augmented copies of one recording never appear in two folds.
- 5 Evaluate each tuned model at **clip level** on the validation split using the same `aggregate_scores()` as the app; select by validation macro-F1, then minimum critical-class recall.
- 6 Evaluate the selected model **once** on the test split; add white noise at 20 / 10 / 5 dB SNR for the robustness report.
- 7 Save the bundle and reports.

Table 17 Training (python_models/train_models.py)

Candidate	Base estimator	Grid
svm	SVC(probability=True, class_weight="balanced")	C ∈ {1, 10, 30}, gamma ∈ {scale, 0.005}
rf	RandomForest(300 trees, class_weight="balanced_subsample")	max_depth ∈ {None, 25}, min_samples_leaf ∈ {1, 2}
xgb	XGBClassifier(300 trees, tree_method="hist")	max_depth ∈ {4, 6}, learning_rate ∈ {0.1, 0.05}
mlp	MLPClassifier((256, 128), early_stopping)	alpha ∈ {1e-4, 1e-3}

8.2 Results of the current model (py-xgb-20260924-0800)

From `reports/python_model_comparison.csv` (validation split, clip level). SVM was not run for this model because fitting it on $\approx 38,000$ segments takes hours.

Table 18 Results of the current model (py-xgb-20260924-0800)

Model	Best parameters	CV macro-F1	Val. accuracy	Val. macro-F1	Train time
XGBoost (selected)	learning_rate 0.1, max_depth 6	0.790	0.844	0.847	1,462 s
MLP	alpha 0.001	0.779	0.805	0.818	292 s
Random Forest	max_depth None, min_samples_leaf 1	0.742	0.769	0.802	953 s

Test split (`reports/python_test_metrics.json`, 695 unseen clips): **accuracy 0.865, macro-F1 0.863**, macro precision 0.847, macro recall 0.895.

Table 19 Results of the current model (py-xgb-20260924-0800) – Class

Class	Precision	Recall	F1
Machinery Fault	0.769	1.000	0.870
Glass Breaking	0.556	0.833	0.667
Alarm or Siren	0.929	0.915	0.922
Vehicle Horn	1.000	0.873	0.932
Animal Sound	0.885	0.793	0.836
Gunshot	0.962	0.927	0.944
Panic Scream	0.636	0.933	0.757
Aggression	0.889	0.889	0.889
Person Asking for Help	1.000	1.000	1.000
Background Noise	0.843	0.782	0.812

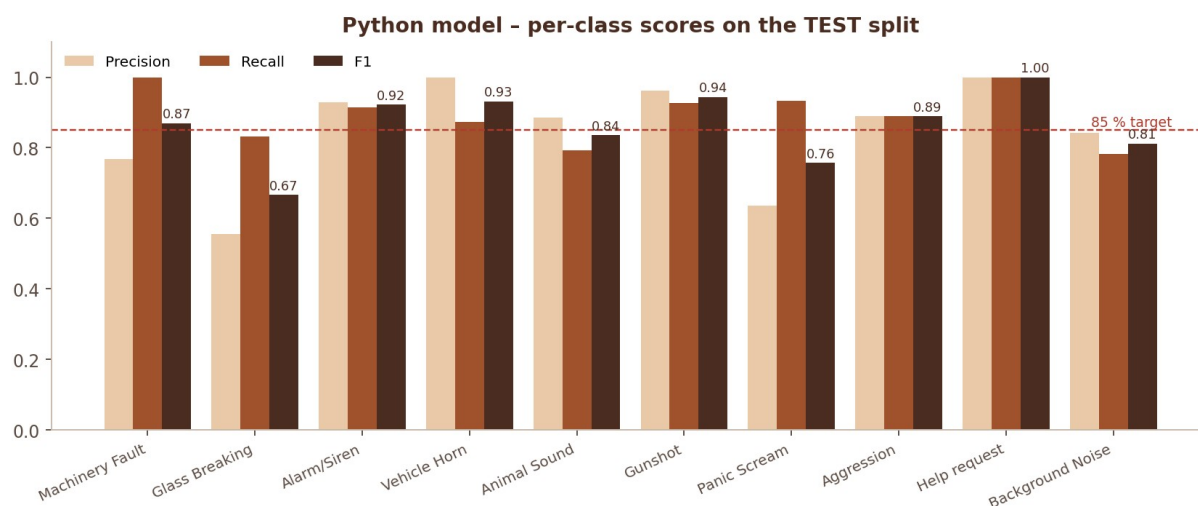


Figure 4 Python model - precision, recall and F1 per class (TEST split)

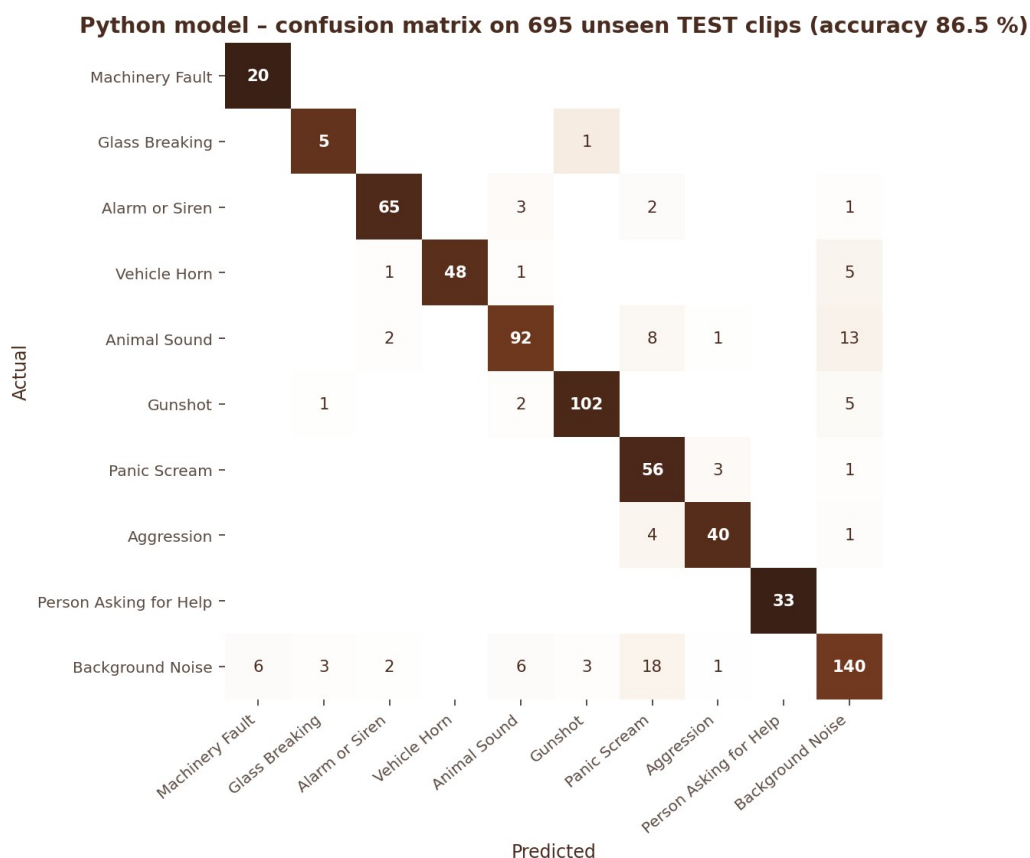


Figure 5 Confusion matrix of the Python model on 695 unseen TEST clips

Table 20 Results of the current model (py-xgb-20260924-0800) – Noise condition (reports/noise_robustness.csv)

Noise condition (reports/noise_robustness.csv)	Accuracy	Macro-F1
clean	0.865	0.863
white noise 20 dB SNR	0.809	0.780
white noise 10 dB SNR	0.711	0.602
white noise 5 dB SNR	0.663	0.529

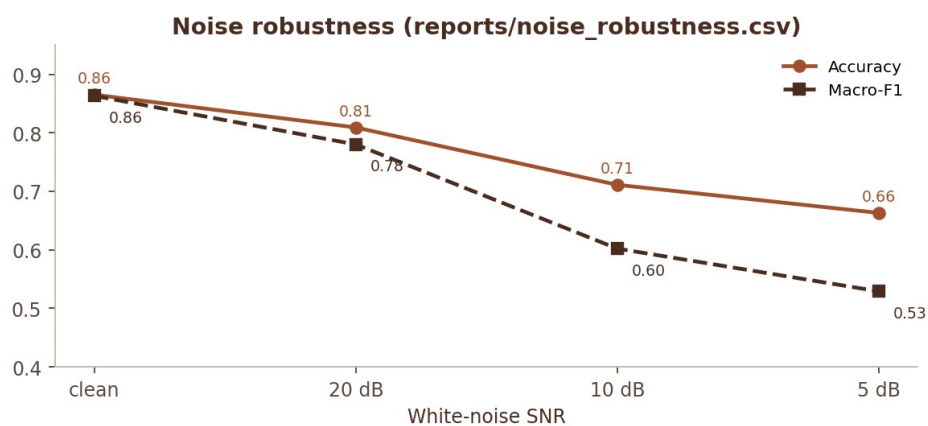


Figure 6 Accuracy and macro-F1 as white noise gets stronger

8.3 Model bundle and inference (python_models/inference.py)

```
python_models/saved/sonic_model.joblib PYTHON
1 bundle = {
2     "pipeline": Pipeline([StandardScaler, XGBClassifier]), "classes": CLASSES,
3     "algorithm": "xgb", "version": "py-xgb-YYYYMMDD-HHMM", "feature_names": [...299],
4     "trained_at": iso-time,
5     "settings": {"segment_seconds", "segment_hop_seconds", "noise_reduction",
6     "sample_rate"},
7     "metrics": {"validation": {...}, "test": {...}, "noise_robustness": [...]}
8 } # joblib.dump -> python_models/saved/sonic_model.joblib
```

- `PythonSoundModel(path)` loads the bundle (raises `ModelNotAvailable` if missing); `predict_proba(X)` returns one `{class: probability}` dict per segment covering all `CLASSES`.
- `analysis.python_model()` caches one instance per process; `warm_up()` runs the pipeline once on synthetic audio at start-up so the first real request does not pay librosa / numba JIT compilation.
- `top_k(scores, k)` and `top_margin(scores)` are shared helpers.

8.4 Clip-level aggregation

`aggregate_scores(segment_scores, min_conf)` is used for **both** models, in training evaluation and in the app:

- If any segment has a non-background top class with confidence $\geq$ `min_conf`, the clip takes the scores of the **strongest such segment** (a 0.3-s gunshot in a 20-s recording must not be averaged away). Its index is stored as `python_segment_index`.
- Otherwise the clip takes the mean over all segments (index -1).



CHAPTER 09

Google Teachable Machine integration

How the Teachable Machine model is trained, exported and run in the browser.

SRS Steps 9–10 · FR xxvii–xxx

9.1 Model format

Teachable Machine audio projects export a TensorFlow.js **speech-commands BROWSER_FFT** model. `gtm_model/model/metadata.json` of the installed export:

```
gtm_model/model/metadata.json  JSON
1  {"tfjsSpeechCommandsVersion":"0.4.0","modelName":"TMv2","timeStamp":"2026-09-
24T20:51:32.771Z",
2  "wordLabels":["Aggression","Alarm or Siren","Animal Sound","Background Noise","Glass
Breaking",
3  "Gunshot","Machinery Fault","Panic Scream","Person Asking for
Help","Vehicle Horn"]}
```

Labels are mapped **by name**, so their order does not matter, but the spelling must match **CLASSES**. `analysis.gtm_info()` reads the metadata, reports missing and extra labels (shown on the dashboard and Admin → Models) and builds the version string `gtm-<modelName>-<timeStamp>` stored with every prediction. The files are served by `GET /gtm/<filename>`.

9.2 Spectrogram computation (static/js/gtm.js)

The TM recorder feeds the model spectrograms from a Web Audio **AnalyserNode**. To classify **uploaded files and live windows** (not only TM's own microphone stream), `gtm.js` reproduces that front end exactly:

- 44.1 kHz mono audio, FFT size 2048, Blackman window, magnitude in dB (`20·log10(|X|/2048)`), first 232 bins.
- 43 frames with a hop of 1,024 samples (≈ 1 s per example), then z-normalisation over the whole 43×232 tensor (as speech-commands `normalize()`).
- `classify(samples)` slides 1-s examples with a 0.5-s hop over the segment and averages the scores; `decodeUrl(url)` decodes a WAV into 44.1 kHz mono with an `OfflineAudioContext`.
- Libraries: `@tensorflow/tfjs@1.3.1` and `@tensorflow-models/speech-commands@0.4.0` (the versions TM exports for), loaded in `templates/partials/gtm_scripts.html`.

9.3 Training data for GTM (gtm_model/prepare_gtm_samples.py)

GTM is trained only on the **TRAIN split** of the same dataset. The `--zip` mode writes one archive per class in Teachable Machine's own sample format, so samples can be uploaded digitally instead of being replayed through speakers:

```

● ● ● prepare_gtm_samples.py – output
1 python -m gtm_model.prepare_gtm_samples --max-per-class 400 --zip
2 -> gtm_model/training_samples/_tm_upload/<Class>.zip
3 samples.json [{ "frequencyFrames": [43 x 232], "blob": null, "blobFilePath":
"sample-1.webm",
4 "startTime", "endTime", "recordingDuration" }, ...]
5 sample-1.webm all samples one after another (Opus via FFmpeg; playback only)
6 -> gtm_sample_counts.csv, gtm_samples_metadata.csv (Audio ID of every source
recording)

```

The spectrogram frames are computed with the same FFT 2048 / Blackman / hop-1024 parameters as `gtm.js.gtm_model/tm_upload_fix.js` is a console snippet that works around two Teachable Machine web-app bugs on Windows: `.zip` files reported as `application/x-zip-compressed` are rejected by the upload box, and `String.fromCharCode.apply` overflows the call stack when thousands of examples are loaded.

9.4 GTM results

Table 21 GTM results

Run	Export	Test accuracy	Macro-F1	Notes
2	2026-09-24 15:50	0.158	0.174	under-trained export (≈ 20 % on its own training data)
3 (installed)	2026-09-24 20:51	0.606	0.607	best: Alarm or Siren F1 0.82, Help 0.73, Horn 0.72; weakest: Panic Scream 0.41
Combined with Python	<code>python_weight</code> 0.7	0.868	0.866	better than either model alone

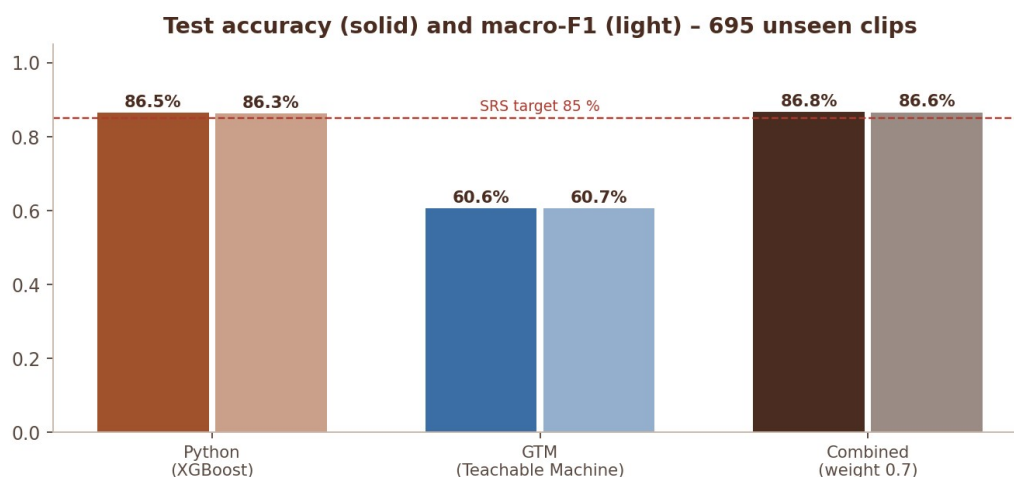


Figure 7 Python, GTM and combined decision against the 85 % SRS target

A classification head trained with SGD on the same TM base model reaches ≈ 0.64, so ≈ 0.6 – 0.65 is the practical ceiling of the 1-second speech-commands transfer model on these environmental sounds. This is documented as a known limitation ([chapter 19](#)).



CHAPTER 10

Decision engine

The pure-function decision engine: comparison, combination, rules, look-alikes and review routing.

SRS Steps 11, 14-17, 19 · FR xxxi-lji, lvii

`src/services/decision.py` contains only **pure functions** (no database, no Flask). This makes the decision logic easy to unit-test (`tests/test_decision_rules.py`) and easy to explain in a viva. `analysis.finalize()` prepares the inputs and stores the outputs.

10.1 Inputs

```

src/services/decision.py
1 decide(py, gtm, *, quality, noise_db, repeated_for, rules, settings) -> dict
2   py, gtm      clip-level {class: score} from aggregate_scores() (gtm may be None)
3   quality      "Good" | "Acceptable" | "Poor" | "Unusable"
4   noise_db     estimated noise floor (dBFS)
5   repeated_for function(category) -> consecutive detections
6               uploads: longest run of consecutive segments whose (Python+GTM)/2 top
class = category
7               live:    consecutive windows of the same session inside
repeat_window_seconds
8   rules        load_rules(); settings = get_settings()

```

10.2 Algorithm

- compare_models()** – top class, confidence, top-2 margin and top-3 of each model; `class_match`, `confidence_diff = |py_conf - gtm_conf|`; consistency status: *Uncertain Result* if both < `min_confidence` or GTM missing, *Acceptable Match* if same class, both ≥ `min_confidence` and `diff ≤ acceptable_match_diff`, *Weak Match* if same class otherwise, else *Model Disagreement*.
- combine()** – $w \cdot py + (1 - w) \cdot gtm$ per class with $w = python_weight$ (Python only if GTM is missing). Best class, confidence and margin are taken from the combined vector.
- Unknown** – combined confidence < `unknown_threshold` → category "Unknown".
- Overlap** – ≥ 2 non-background classes with combined score ≥ `overlap_threshold`.
- Uncertain reasons** – below `min_confidence`, margin < `top2_margin`, Poor / Unusable quality, models disagree, GTM unavailable, overlap.
- Rule evaluation** – `rules.evaluate_rule()` checks confidence, top-2 margin, quality (`quality_ok`), model agreement (if required) and repeated detection (`repeated ≥ required_consecutive`, or a single detection when both models agree at ≥ `strong_confidence`). All checks true = *confirmed*. Severity is raised to `escalate_to` when `repeated ≥ escalate_after_repeats`.
- Alert status** – Background Noise with `noise_db > background_noise_limit_db` → noise alert. Otherwise, if `generate_alert`: confirmed → "Alert Generated"; not confirmed → "Pending Confirmation" plus a review reason when `manual_review_if_unconfirmed`.
- Possible false alarm** – an alert where one model gives the class < 0.20 is also sent to review.

- 9 **Look-alike check** – for every pair in `LOOKALIKE_PAIRS[category]`: `gap = combined[category] - combined[look-alike]`; `gap < lookalike_margin` → review "Possible look-alike: could be ...". For a Background Noise decision, any critical class whose look-alike is Background and whose score $\geq$ `unknown_threshold` adds "Possible <class> that sounds like ...".
- 10 **Review routing** – disagreement, low confidence, poor quality, close top-2, overlap, Unknown. Clear background noise keeps only the disagreement and "Possible ..." reasons. Duplicates are removed.
- 11 **Status** – Manual Review > Alert Generated > Uncertain > Classified. Every step appends a line to `trace`, which is stored as `decision_trace` and shown on the event page.

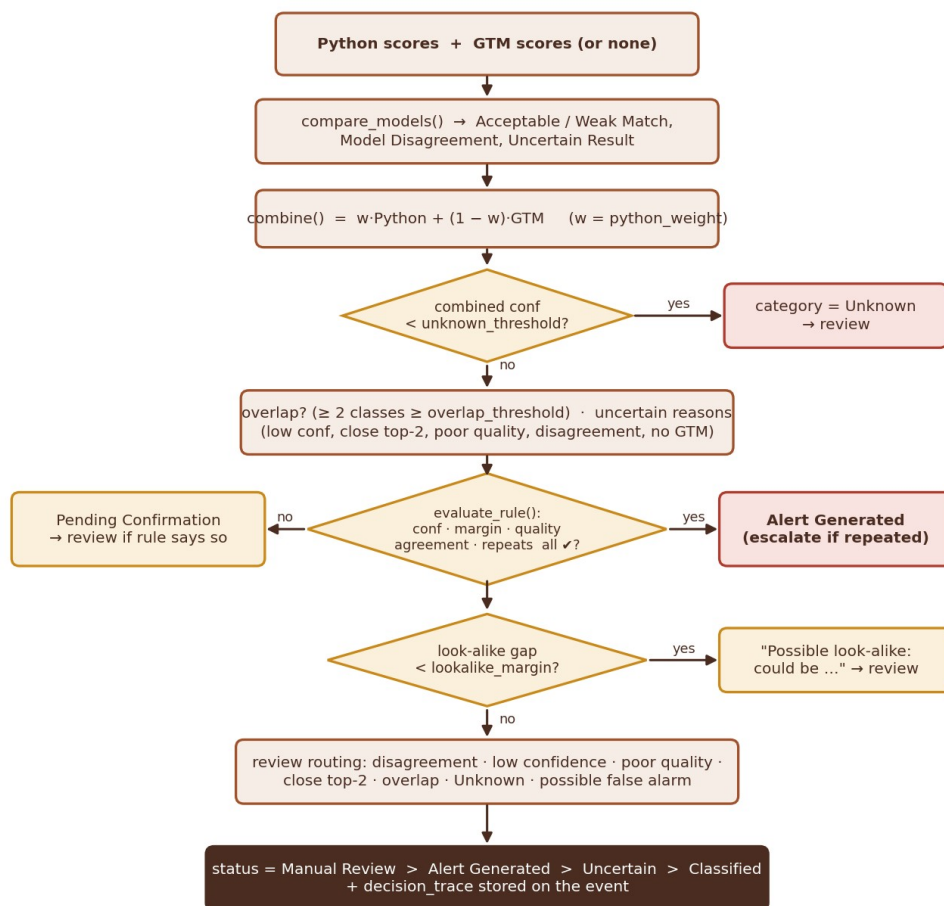


Figure 8 Decision flow inside `decide()`

10.3 Output

The returned dict contains all comparison fields plus `combined_scores`, `final_category`, `final_confidence`, `combined_margin`, `confidence_level` (High ≥ 0.85 / Medium $\geq$ `min_confidence` / Low), `overlap_detected`, `overlap_classes`, `uncertain`, `uncertain_reasons`, `repeated_count`, `severity`, `alert_status`, `recommended_action`, `manual_review_required`, `review_reasons`, `status`, `rule_checks`, `audience`, `lookalike_checks`, `trace`. `finalize()` copies them onto `AudioEvent` and creates an `Alert` (message e.g. "Critical – Gunshot detected (91%)") when `alert_status == "Alert Generated"`.

10.4 Example trace

● ● ● AudioEvent.decision_trace – example

TEXT

```
1 Python → Panic Scream (0.81); GTM → Panic Scream (0.58); |Δconf| = 0.23 → Weak Match
2 Combined scores (Python 70% / GTM 30%) → Panic Scream (0.74), top-2 margin 0.61
3 Rule 'Panic Scream': confidence=✓, top2_margin=✓, quality=✓, model_agreement=✓,
repeated_detection=✓ (repeated 1x)
4 All conditions met → Critical alert
5 Look-alike check: Panic Scream vs normal shouting or loud voices (Background Noise) –
gap 0.66 ≥ 0.20 → distinguished
6 Final: Panic Scream | severity Critical | Alert Generated | status Alert Generated
```



CHAPTER 11

Data model

Tables, relationships, identifiers and the event life cycle.

SRS Step 20 · FR lxii, lxiii, lxv

11.1 Entity relationships

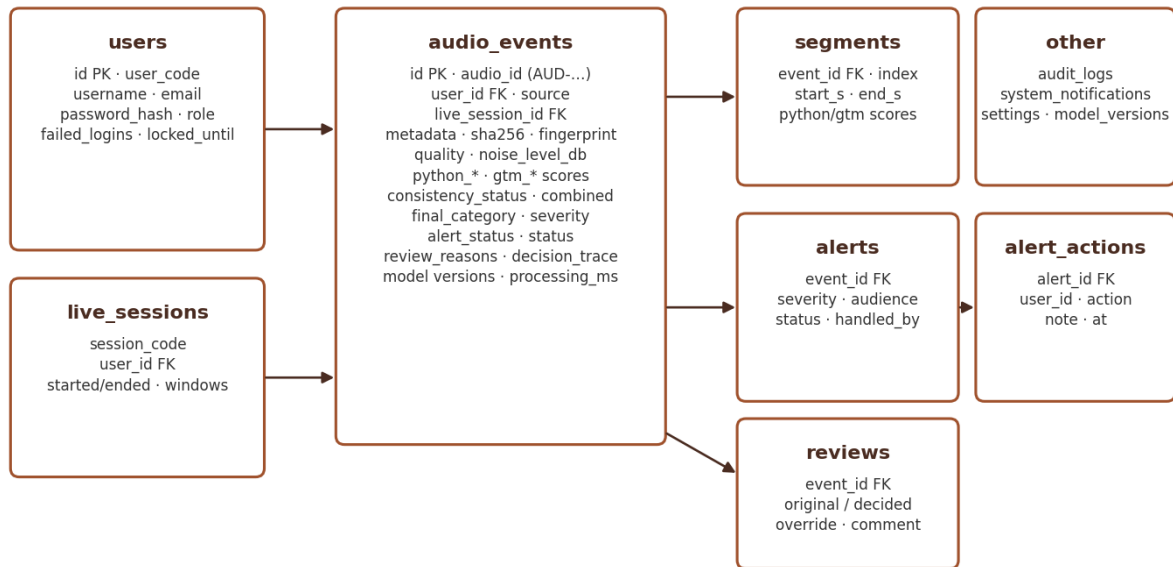


Figure 9 Entity-relationship overview

Models are defined in `src/models.py` (Flask-SQLAlchemy). `JSONText` is a `TypeDecorator` that stores dicts and lists as JSON text, so the schema stays portable to PostgreSQL. `now()` returns a naive UTC-local timestamp used for all defaults.

Table 22 Entity relationships

Table	Key columns	Notes
<code>users</code>	user_code (USR-00001), username, email, password_hash, role, is_active_flag, failed_logins, locked_until, last_login	Werkzeug password hashing; <code>has_role(*roles)</code> lets admin pass every check
<code>live_sessions</code>	session_code, user_id, started_at, ended_at, windows	one per Start ... Stop
<code>audio_events</code>	audio_id, source (upload batch live evaluation), actual_class, file metadata, sha256, fingerprint, duplicate / near-duplicate links, quality, python_* / gtm_* (prediction, confidence, scores, margin), gtm_status, class_match, consistency_status, confidence_diff, combined_scores, overlap, uncertain, repeated_count, final_*, severity, alert_status, recommended_action, review fields, status, decision_trace, model versions, processing_ms	central table; indexes on audio_id, user_id, uploaded_at, final_category, severity, status
<code>segments</code>	event_id, index, start_s, end_s, audio_path (44.1 kHz WAV), rms_db, python / gtm scores and predictions	cascade delete with the event

Table	Key columns	Notes
<code>alerts</code>	<code>event_id</code> , <code>category</code> , <code>severity</code> , <code>message</code> , <code>recommended_action</code> , <code>audience</code> (JSON roles), <code>status</code> , <code>handled_by_id</code> , <code>handled_at</code>	status: Active Acknowledged Escalated Dismissed
<code>alert_actions</code>	<code>alert_id</code> , <code>user_id</code> , <code>action</code> , <code>note</code> , <code>at</code>	history of every operator action
<code>reviews</code>	<code>event_id</code> , <code>reviewer_id</code> , <code>original_category</code> , <code>decided_category</code> , <code>decision</code> (Confirmed Corrected), <code>override</code> , <code>comment</code> , <code>recommended_action</code>	original model output is never overwritten
<code>audit_logs</code>	<code>user_id</code> , <code>username</code> , <code>action</code> , <code>entity</code> , <code>entity_id</code> , <code>details</code> , <code>ip</code> , <code>at</code>	written by <code>audit.log()</code>
<code>system_notifications</code>	<code>kind</code> , <code>message</code> , <code>at</code> , <code>is_read</code>	admin anomaly alerts
<code>settings</code>	<code>key</code> , <code>value</code> (JSON), <code>updated_at</code>	runtime overrides of <code>DEFAULT_RUNTIME_SETTINGS</code>
<code>model_versions</code>	<code>model_type</code> (python gtm), <code>version</code> , <code>details</code> , <code>registered_at</code>	registered on first use of a new version

11.2 Event life cycle

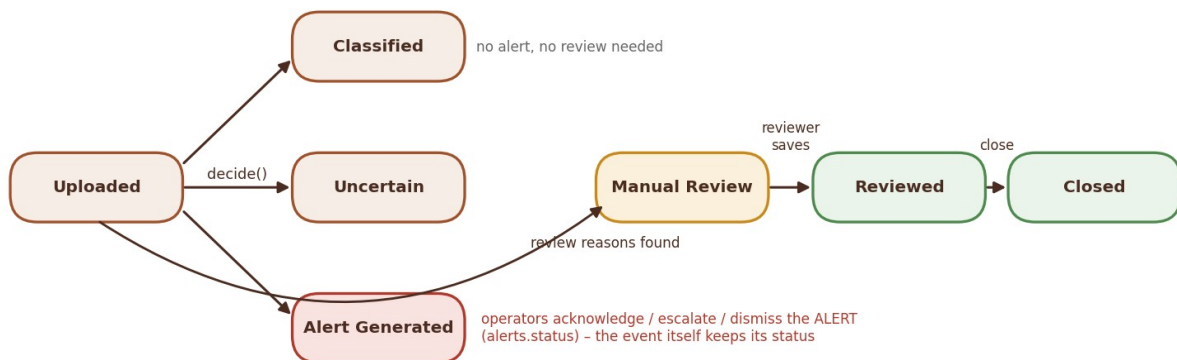


Figure 10 Event status life cycle (`EVENT_STATUSES`)

IDs: events `AUD-YYYYMMDD-HHMMSS-XXXX` (4 hex characters), live windows `LIV-....`, sessions `LS-....`. Files: `data/uploads/` (originals), `data/segments/<audio_id>/seg_NNN.wav`, `data/images/`. `database/init_db.py` → `seed_defaults()` creates the five default accounts if they do not exist.



CHAPTER 12

Web application

Application factory, security, routes and the front end – including an annotated tour of the event page.

SRS FR i-iii, lxiii-lxvii, lxxi, lxxvii, lxxix

12.1 Application factory (src/__init__.py)

- `create_app(test_config=None)` configures Flask (secret key, SQLite URI, `MAX_CONTENT_LENGTH = 25 MB × 20` for batch requests, `HttpOnly + SameSite=Lax` cookies), initialises SQLAlchemy and Flask-Login, registers six blueprints and the CSRF hook.
- A context processor injects `csrf_token`, `CLASSES`, `SEVERITY_LEVELS`, `ROLES` and the navigation badge counts (active alerts, unread notifications, pending reviews) according to the role.
- Template filters: `pct` (`0.742` → `"74.2%"`) and `dt`.
- Error handlers for 400, 401, 403, 404, 413 and 500 return JSON for `/api/*` and `errors/error.html` otherwise; 500 rolls back the session.
- On start-up: `db.create_all()`, `seed_defaults()`, `apply_retention()` and `warm_up()` (skipped in tests).
- `GET /healthz` returns `{"status", "uptime_s", "checks": {database, python_model, gtm_model}}` with HTTP 200, or 503 if the database fails.

12.2 Security (src/security.py)

Table 23 Security (src/security.py)

Mechanism	Implementation
Authentication	Flask-Login sessions; login by username or email; "remember me" cookie
Password policy	≥ 8 characters with letters and digits (<code>password_problems</code>)
Lock-out	5 failed attempts → <code>locked_until = now + 5 min</code> ; each failure audited; ≥ 3 failures in 15 min → admin notification
CSRF	per-session token (<code>secrets.token_hex(16)</code>), checked in <code>before_request</code> for POST / PUT / PATCH / DELETE; forms send <code>csrf_token</code> , fetch sends <code>X-CSRFToken</code> ; compared with <code>secrets.compare_digest</code>
Authorisation	<code>@roles_required(*roles)</code> (admin always allowed); admin blueprint guarded in <code>before_request</code> ; <code>can_view_event()</code> limits Normal Users to their own events; <code>can_handle_alert()</code> uses the alert audience
Media	audio and images are served only through routes that call <code>can_view_event()</code> ; Lab files are checked against the owner of the token
Open redirect	<code>next</code> parameter accepted only when it starts with <code>"/"</code> and not <code>"/"</code>

**SECURITY Rule of thumb for new routes**

Every new route needs `@login_required`, a role check (`@roles_required`) where staff-only, and `can_view_event()` for anything that returns event data or audio. POST handlers get CSRF protection automatically; fetch calls must go through `SS.api()`.

12.3 Page routes

Table 24 Page routes

Route	Methods	Access	Purpose
<code>/register</code> , <code>/login</code> , <code>/logout</code> , <code>/profile</code>	GET, POST	public / logged in	accounts (blueprint <code>auth</code>)
<code>/</code> , <code>/dashboard</code>	GET	logged in	dashboard (class counts, KPIs, lists)
<code>/upload</code>	GET, POST	logged in	upload page / form upload
<code>/batch</code>	GET	reviewer, security, maintenance, admin	batch page (uploads go through <code>/api/upload</code>)
<code>/live</code> , <code>/lab</code>	GET	logged in	live monitoring, Robustness Lab
<code>/events/<audio_id></code>	GET	owner or staff	event page
<code>/events/<audio_id>/report</code>	GET	owner or staff	printable report
<code>/media/event/<audio_id>/<kind></code>	GET	owner or staff	audio, waveform, spectrogram, explain image
<code>/media/segment/<seg_id></code>	GET	owner or staff	segment WAV (GTM input, reviewer playback)
<code>/gtm/<filename></code>	GET	logged in	GTM model files
<code>/history</code> , <code>/timeline</code>	GET	logged in	filtered list (paged) / chronological view
<code>/alerts/</code> , <code>/alerts/history</code>	GET	security, maintenance, admin	alert list and history
<code>/alerts/<id>/<acknowledge dismiss escalate></code>	POST	security, maintenance, admin (+ audience)	alert action (JSON for fetch)
<code>/review/</code> , <code>/review/<audio_id></code>	GET, POST	reviewer, admin	queue, review form
<code>/admin/...</code>	GET, POST	admin	dashboard, users, settings, retention, rules, audit, notifications, models, confusion matrix, evaluation, export
<code>/healthz</code>	GET	public	health check

12.4 Front end

- `templates/base.html` loads Tailwind from the CDN with a custom `tailwind.config` (Mocha palette), `static/css/app.css` (components: cards, badges, `.hero`, `.kpi`, `.class-tile`, `.cls-chip`, `.auth-shell`, `.dropzone` ...) and `app.js`.

- `app.js` exposes the global `SS` helper: `SS.api()` (fetch with CSRF header and JSON errors), `SS.toast()`, badge renderers, alert polling (`/alerts/poll`) for toasts.
- `partials/macros.html` (badges, `event_row`), `partials/icons.html` (original stroke-SVG icon set and one icon per sound class: `icon()`, `class_icon()`, `class_chip()`), `partials/gtm_status.html`, `partials/auth_art.html`.
- `wav.js` encodes Float32 audio as 16-bit PCM WAV for live windows; `batch.js` uploads files one by one and runs GTM on each; `lab.js` drives the Robustness Lab.

12.5 Anatomy of the event page

The event page (`templates/events/detail.html`) shows every stage of the pipeline for one recording. The numbers in the screenshot map each block to the code that produces it.

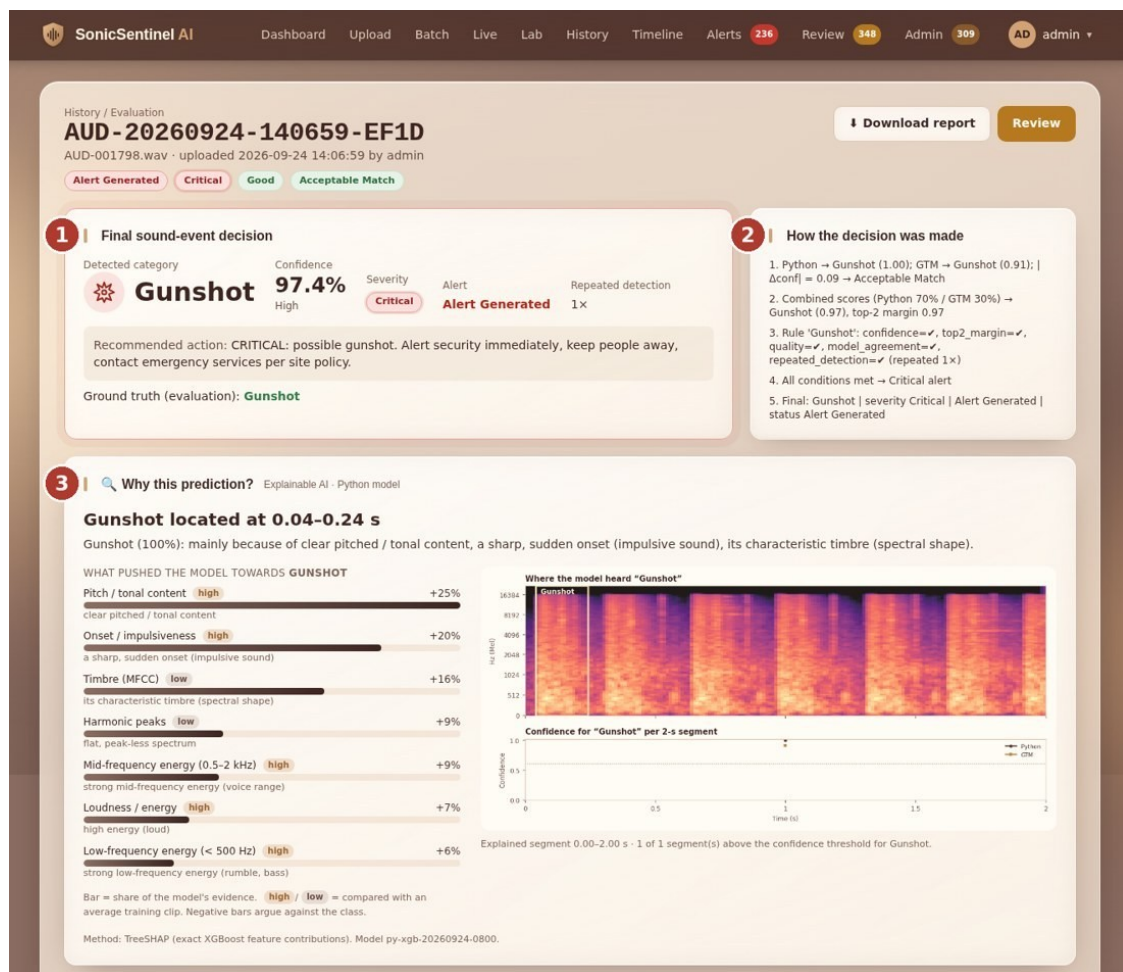


Figure 11 Event page (blocks 1-3) – a correctly detected Gunshot: Acceptable Match, Critical alert, TreeSHAP explanation

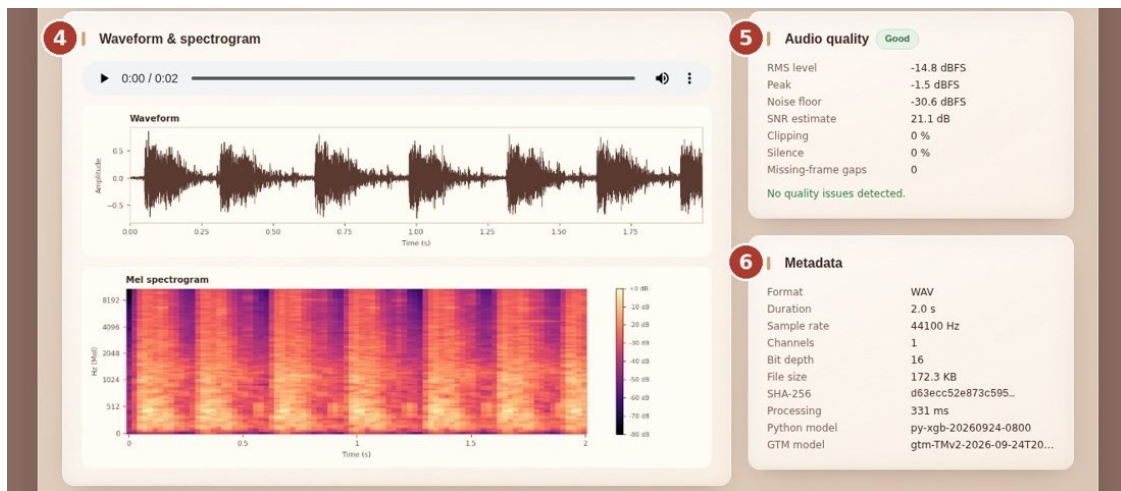


Figure 12 Event page (blocks 4-6) – waveform, spectrogram, audio quality and metadata

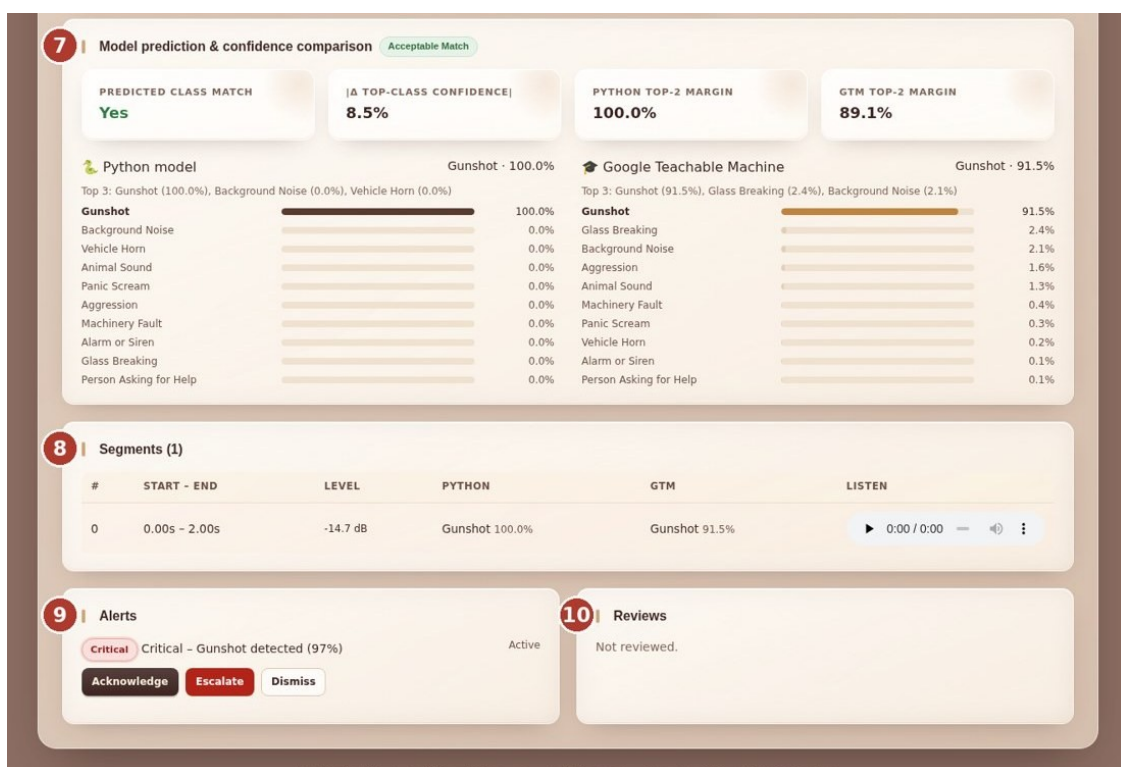


Figure 13 Event page (blocks 7-10) – model comparison, segments, alerts and reviews

Table 25 Event page blocks and the code behind them

#	Block	Produced by	Data
1	Final sound-event decision	<code>analysis.finalize()</code> → <code>decision.decide()</code>	<code>final_category</code> , <code>final_confidence</code> , <code>confidence_level</code> , <code>severity</code> , <code>alert_status</code> , <code>recommended_action</code> , <code>repeated_count</code>
2	How the decision was made	<code>decide()</code> trace lines	<code>decision_trace</code> (JSON list)
3	Why this prediction? (XAI)	<code>GET /api/events/<id>/explain</code> → <code>services/explain.py</code>	TreeSHAP groups, span, <code><audio_id>_explain.png</code>
4	Waveform & spectrogram	<code>feature_extraction/visuals.py</code>	<code>/media/event/<id>/audio waveform spectrogram</code>

#	Block	Produced by	Data
5	Audio quality	<code>audio_preprocessing/quality.py</code> → <code>analyze_quality()</code>	<code>quality JSON</code> , <code>quality_label</code> , <code>noise_level_db</code>
6	Metadata	<code>AudioData.metadata()</code> , <code>sha256_file()</code> , model registry	format, duration, SR, channels, bit depth, size, SHA-256, versions, <code>processing_ms</code>
7	Model prediction & confidence comparison	<code>compare_models()</code> ; GTM scores from <code>gtm.js</code>	<code>python_scores</code> , <code>gtm_scores</code> , <code>class_match</code> , <code>confidence_diff</code> , margins, top-3
8	Segments	<code>analyze_upload()</code> segmentation	<code>segments</code> rows, <code>/media/segment/<id></code>
9	Alerts	<code>finalize()</code> ; POST <code>/alerts/<id>/<action></code>	<code>alerts</code> , <code>alert_actions</code>
10	Reviews	<code>/review/<audio_id></code>	<code>reviews</code> (original output preserved)



CHAPTER 13

JSON API reference

Every JSON endpoint with its request and response.

SRS FR iv–vii, xxix, liv

All endpoints are under `/api`, require a logged-in session and, for POST, the `X-CSRFToken` header. Errors are returned as `{"error": "message"}` with 400, 403, 404, 422 or 500.

Table 26 JSON API reference

Endpoint	Request	Response
POST <code>/api/upload</code>	multipart <code>audio</code> file	<code>{event, warnings}; {duplicate: true, event, message}; 422 {error}</code> on validation failure. Source is "batch" for staff, "upload" for users
GET <code>/api/events/<audio_id></code>	-	event JSON with segments
POST <code>/api/events/<audio_id>/gtm</code>	<code>{"segments": [{class: score}, ...]}</code> (one per segment) or <code>{"error": "..."}</code>	final event JSON; 400 if the segment count differs; idempotent once GTM is done
GET <code>/api/events/<audio_id>/explain</code>	-	<code>{target, confidence, headline, summary, against, span, segment_used, segments, groups, method, model_version, image_url, ...}</code>
POST <code>/api/live/start</code>	-	<code>{session}</code>
POST <code>/api/live/window</code>	multipart: <code>session</code> , <code>audio</code> (WAV), <code>gtm</code> (JSON scores) or <code>gtm_error</code>	event JSON + <code>alert</code> or <code>{silent: true, quality, message}</code>
POST <code>/api/live/stop</code>	<code>{"session": code}</code>	<code>{ok: true}</code>
GET <code>/api/live/recent</code>	-	last 15 live events
GET <code>/api/stats</code>	query <code>days</code> (default 14)	chart data for dashboards
GET <code>/api/evaluate/testset</code>	admin; query <code>per_class</code> (default 10)	list of unseen TEST clips to evaluate
POST <code>/api/evaluate/file</code>	admin; <code>{"audio_id": "AUD-..."}</code> of a TEST clip	analyses one test clip as source "evaluation" with <code>actual_class</code>
POST <code>/api/lab/load</code>	multipart <code>audio</code> or <code>{class}</code> for a random test clip	<code>{token, meta}</code>
POST <code>/api/lab/run</code>	<code>{token, params}</code>	Python result + URL of the degraded 44.1 kHz WAV for GTM
POST <code>/api/lab/sweep</code>	<code>{token, params, kind}</code>	confidence curve over 7 noise levels
GET <code>/api/lab/audio/<name></code>	-	degraded WAV (owner only)

Event JSON fields: `audio_id`, `url`, `filename`, `source`, `status`, `uploaded_at`, `duration`, `quality`, `gtm_status`, `python_prediction`, `python_confidence`, `gtm_prediction`, `gtm_confidence`, `python_scores`, `gtm_scores`, `consistency_status`, `confidence_diff`, `final_category`, `final_confidence`, `confidence_level`, `severity`, `alert_status`, `recommended_action`, `manual_review`, `review_reasons`, `repeated_count`, `overlap`, `overlap_classes`, `trace`, `actual_class`, `processing_ms` and, with segments, `segments: [{id, index, start, end, url}]`.



CHAPTER 14

Supporting services

Audit, analytics, explainable AI, Robustness Lab, retention and settings.

SRS FR lxxviii–lxx, lxxvi, lxxviii, lxxx

Table 27 Supporting services

Module	Functions	Details
<code>audit.py</code>	<code>log()</code> , <code>notify()</code> , <code>check_failed_uploads()</code> , <code>check_failed_logins()</code> , <code>check_after_event()</code>	audit rows with IP; notifications de-duplicated for 10 min. Triggers: ≥ 3 rejected uploads in 10 min, ≥ 3 failed logins in 15 min, ≥ 6 of the last 10 detections below 0.50, ≥ 10 critical alerts in 10 min, duplicates, model and report failures
<code>analytics.py</code>	<code>summary(events)</code> , <code>comparison_dataframe(events)</code>	dashboard KPIs, FP / FN from reviews and evaluation, response times; one row per event with every SRS comparison column + summary sheet
<code>explain.py</code>	<code>explain_event(ev, model)</code> , <code>explain_vector()</code>	XGBoost: exact TreeSHAP (<code>pred_contribs=True</code>) summed over 13 feature groups; other models: group occlusion in log-odds. Locates the loud span inside the deciding segment and draws <code><audio_id>_explain.png</code>
<code>lab.py</code>	<code>load_file()</code> , <code>load_test_clip()</code> , <code>degrade()</code> , <code>python_analyse()</code> , <code>run()</code> , <code>sweep()</code>	uses the training augmentation functions; tokens and WAVs in <code>data/lab/</code> cleaned after 3 h; nothing written to the event tables
<code>retention.py</code>	<code>apply_retention()</code>	deletes audio older than <code>audio_retention_days</code> (record kept) and records older than <code>record_retention_days</code>
<code>settings_service.py</code>	<code>get_settings()</code> , <code>update_settings()</code>	defaults merged with DB overrides; type-safe casting
<code>rules.py</code>	<code>load_rules()</code> , <code>save_rules()</code> , <code>default_rule()</code> , <code>validate_rule()</code> , <code>evaluate_rule()</code> , <code>quality_ok()</code>	alert-rule file handling and evaluation

The 13 explanation groups are: onset / impulsiveness, loudness, brightness, frequency spread, noisiness, low- (< 500 Hz), mid- (0.5 – 2 kHz) and high-frequency (> 2 kHz) energy, timbre (MFCC), timbre change (Δ -MFCC), pitch / tonal content, harmonic peaks and rhythm.



CHAPTER 15

Testing

Automated tests, benchmarks and hidden-test evidence.

SRS SRS 1.8 hidden tests · NFR 1-2 · Deliverable 8

15.1 Test suite

Run `python -m pytest -q`. The suite has **70 tests** and runs in isolation: `tests/conftest.py` sets `SONIC_DATA_DIR`, `SONIC_DB_PATH`, `SONIC_MODEL_PATH`, `SONIC_GTM_DIR` (empty → GTM "not installed") and `SONIC_RULES_PATH` to a temporary folder **before** the app is imported, and trains a tiny Random Forest on synthetic tones (one frequency per class). Your real database, model and rules are never touched.

Table 28 Test suite

File	Tests	What is covered
<code>test_app.py</code>	18	login required, CSRF, wrong password and lock-out, registration and User ID, role access, full upload flow with posted GTM scores, disagreement → review, invalid / silent uploads, duplicates, low quality, live-window flow, repeated live Gunshot confirmation, every page renders, exports, admin settings and rules, audit trail, database tables, /healthz
<code>test_audio_pipeline.py</code>	22	WAV / FLAC / OGG / MP3 (FFmpeg) loading, unsupported / empty / corrupt / silent / too short files, stereo metadata, resampling, normalisation, segmentation and padding, silence trimming, every quality rule, feature vector length and finiteness, augmentations, near-duplicate detection of trimmed and quieter copies
<code>test_decision_rules.py</code>	20	Acceptable Match → critical alert, $ \Delta\text{conf} $ formula, Gunshot confirmation, disagreement, low confidence and Unknown, overlap, close top-2, poor quality blocks alerts, background quiet vs loud, non-critical logged only, Aggression escalation, GTM unavailable, false alarm, rule validation, strong-confidence single detection, short-event aggregation, <code>python_weight</code> , look-alike checks
<code>test_explain_lab.py</code>	8	XAI contributions, explain API and image, login required; Lab page, upload → run → sweep → audio, bad input, clip ownership, deterministic <code>degrade()</code>
<code>test_gtm_samples.py</code>	2	TM spectrogram frame shape and peak, TM zip format

15.2 Writing a new test

```

tests/test_my_feature.py
PYTHON
1 def test_my_feature(client, wav_file):
2     login(client)                                # helper in conftest (admin by
default)
3     path = wav_file(class_signal(5))              # synthetic "Gunshot" tone
4     with open(path, "rb") as f:
5         r = client.post("/api/upload", data={"audio": (f, "clip.wav")},
6                             headers={"X-CSRFToken": csrf(client)})
7     assert r.status_code == 200
8     ev = r.get_json()["event"]
9     assert ev["python_prediction"] == "Gunshot"

```

Decision tests call `decide()` directly with hand-made score dicts – no audio or database needed. Keep new decision rules covered by at least one positive and one negative test.

15.3 Evaluation and benchmark scripts

Table 29 Evaluation and benchmark scripts

Script	Output	Latest result
<code>scripts/ benchmark_performance.py</code>	<code>reports/ performance.md/json</code>	30-s upload 1.47 s mean (max 1.74 s); live window 0.11 s; 20,020 events; dashboard 0.07 s, history 0.18 s, admin 0.48 s, CSV export 1.34 s; 5 concurrent users, 0 errors (2-CPU Linux)
<code>scripts/ hidden_test_robustness.py</code>	<code>reports/ hidden_test_robustness. md/csv</code>	266 unseen TEST clips, 2,926 runs, 0 crashes: clean 0.90, echo 0.87, MP3 re-encode 0.88, low volume 0.90, distant 0.77, other device 0.76, partial 0.75, overlap 0.74, real background 10 dB 0.72, white noise 20 dB 0.71 / 10 dB 0.55
<code>scripts/ lookalike_report.py</code>	<code>reports/ lookalike_pairs.md/csv</code>	recall of each event, false-alarm rate of its look-alike, training coverage (e.g. scream vs voices: 33 % of voices mistaken; backfire: no data yet)
Admin → Model comparison	<code>reports/ model_comparison_report .xlsx</code>	695 TEST clips, both models in the real browser + server path: Python 0.865, GTM 0.606, final 0.868 (weight 0.7)

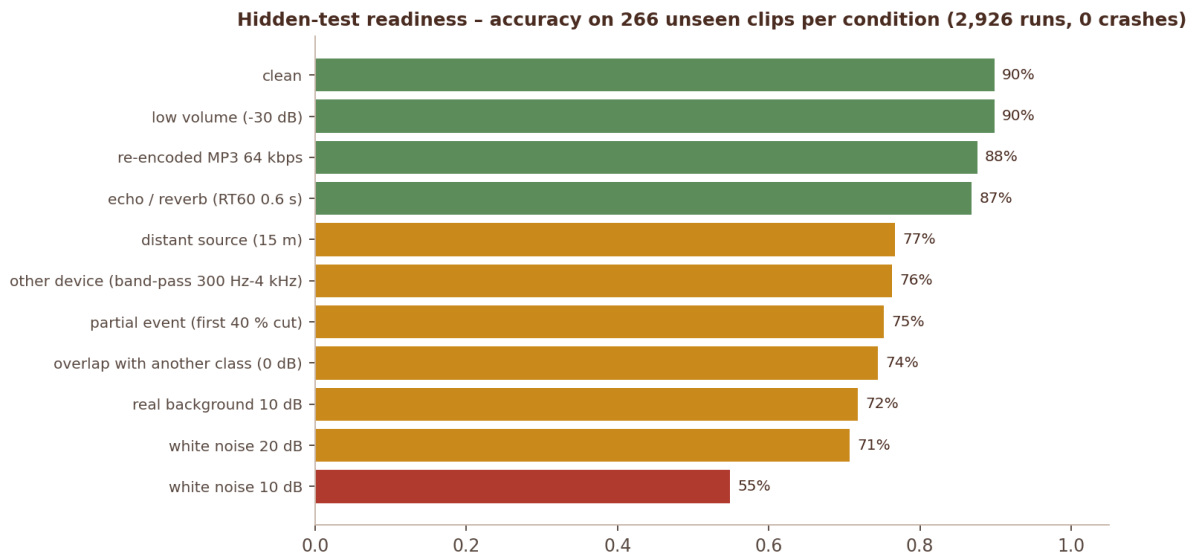


Figure 14 Hidden-test readiness per degradation (reports/hidden_test_robustness.csv)

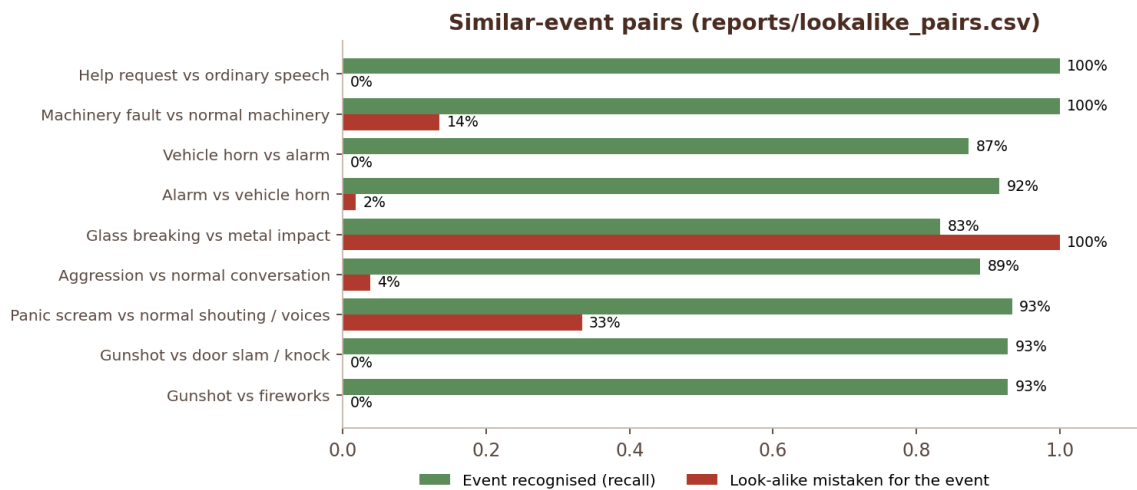


Figure 15 Similar-event pairs - event recall vs. look-alike false alarms



CHAPTER 16

Dataset and data scripts

Building the common dataset and its current state.

SRS Steps 1, 5 · FR xvi-xix · Dataset hint

16.1 Build pipeline

```

Dataset build
POWERSHELL / SHELL

1 # 1. collect clips into audio_dataset/raw/<Class Name>/
2 python scripts/import_public_datasets.py --urbansound8k D:/data/UrbanSound8K --esc50
D:/data/ESC-50-master
3 python scripts/import_local_downloads.py --sources urbansound8k,esc50,mimii,gunshot,...
[--dry-run]
4 python scripts/generate_help_phrases_tts.py --per-phrase 10 # offline TTS,
safety phrases only
5 python scripts/record_samples.py --class "Person Asking for Help" --speaker S01 --count
20
6
7 # 2. validate, de-duplicate, Audio IDs, 44.1 kHz mono WAV, stratified 70/15/15 split
8 python scripts/build_dataset.py [--min-per-class 300] [--seed 42]
9
10 # 3. augment the TRAIN split only
11 python -m augmentation.augment --per-clip 2

```

data/dataset_metadata.csv columns: audio_id, filename, original_filename, class_label, source, license, duration, sample_rate, channels, original_sample_rate, original_channels, environment, device, distance_m, is_augmented, parent_audio_id, augmentation, split, quality, sha256, path. The split is made by original clip, so segments and augmented copies always stay with their parent. Also written: dataset_statistics.json, dataset_quality_report.csv, dataset_rejected.csv. See documentation/DATA_DICTIONARY.md and DATASET_GUIDE.md for sources and licences.

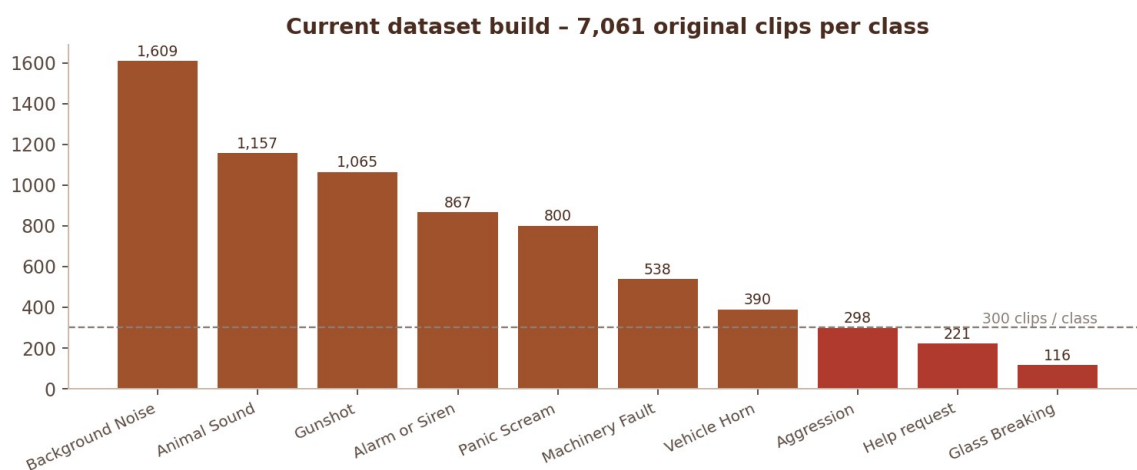
16.2 Current state of the data

Table 30 Current state of the data

Build	Original clips	Split (train / val / test)	Used by
Build used for the installed model	4,629 (≈ 6.6 h)	3,240 / 694 / 695	py-xgb-20260924-0800, GTM run 3, all reports in this guide
Current data/dataset_statistics.json (rebuilt 2026-09-24 21:28 UTC, includes look-alike clips)	7,061 (≈ 10.7 h), 345 rejected	4,942 / 1,059 / 1,060	not yet trained – retrain both models to use it

Table 31 Current state of the data – Class (current build)

Class (current build)	Clips	Class	Clips
Background Noise	1,609	Machinery Fault	538
Animal Sound	1,157	Vehicle Horn	390
Gunshot	1,065	Aggression	298
Alarm or Siren	867	Person Asking for Help	221
Panic Scream	800	Glass Breaking	116

**Figure 16** Clips per class in the current dataset build**WARNING Action for the team**

The dataset was rebuilt after the model was trained, so the files in `audio_dataset/processed/` no longer match the build the model was evaluated on. Before the next evaluation run: `python -m augmentation.augment --per-clip 2` → `python -m python_models.train_models --models xgb,mlp,rf` → regenerate the GTM zips and retrain GTM → Admin → Model comparison. Glass Breaking (116 clips, imbalance ratio 13.9) is still the weakest class.



CHAPTER 17

Deployment

Running SonicSentinel AI on Windows, Docker and Render.

SRS NFR 5 availability · Deliverable 12

17.1 Windows (demo laptop)

- `run_server.bat` – waitress with 8 threads on port 5000, automatic restart if the process exits, log in `logs/server.log`.
- Manual: `waitress-serve --port 5000 run:app`.
- The microphone requires a secure context: open the app on the same machine via `http://127.0.0.1:5000` / `localhost`, or serve it over HTTPS.

17.2 Docker

```

● ● ● Dockerfile DOCKERFILE
1 FROM python:3.11-slim
2 RUN apt-get install ffmpeg libsndfile1 # MP3/M4A decoding, soundfile
3 RUN pip install -r requirements.txt gunicorn
4 ENV PORT=10000 MPLCONFIGDIR=/tmp/matplotlibl NUMBA_CACHE_DIR=/tmp/numba
5 HEALTHCHECK CMD python -c "urllib.request.urlopen('http://127.0.0.1:$PORT/healthz')"
6 CMD python -m database.init_db && gunicorn -w 1 --threads 4 --timeout 180 -b
0.0.0.0:$PORT run:app

```

One worker keeps a single copy of the model in memory; threads handle concurrency. Build with `docker build -t sonic sentinel .` and run with `docker run -p 10000:10000 -e SONIC_SECRET_KEY=... sonic sentinel`.

17.3 Render

`render.yaml` defines a Docker web service `sonic sentinel-ai` with `healthCheckPath: /healthz` and a generated `SONIC_SECRET_KEY`. On the free plan the disk is temporary, so the SQLite database resets on every redeploy – attach a persistent disk (and point `SONIC_DATA_DIR` at it) or switch to PostgreSQL for real use. Full steps: [documentation/DEPLOYMENT.md](#).

17.4 Production checklist

- Set `SONIC_SECRET_KEY` and change all default passwords.
- Serve over HTTPS (required for the microphone from other devices).
- Commit `python_models/saved/sonic_model.joblib` and `gtm_model/model/*` with the release (or mount them).
- Back up `data/` (database, uploads) and configure retention.
- Watch `/healthz` and Admin → Notifications.



CHAPTER 18

Extending the system

Recipes for the most common changes and surprise modifications.

SRS SRS 1.8 surprise modification

18.1 Add a new sound class

- 1 Add the name to `CLASSES` in `config/settings.py` (and to `CRITICAL_RECALL_CLASSES` / `LOOKALIKE_PAIRS` if relevant).
- 2 Create `audio_dataset/raw/<New Class>/` with ≥ 300 licensed or consented clips; rebuild and augment the dataset.
- 3 Retrain the Python model; regenerate GTM zips and add a class with **exactly** the same name in Teachable Machine; export and replace `gtm_model/model/`.
- 4 Add a rule in Admin → Alert rules (or `alert_rules.json`) and an icon in `templates/partials/icons.html` → `_cls` plus a colour in `app.css` (`.cls-<name>`).
- 5 Add decision tests for the new class.

18.2 Common changes

Table 32 Common changes

Change	Where
Confidence threshold, margins, weight, windows	Admin → Settings (<code>DEFAULT_RUNTIME_SETTINGS</code> for new defaults)
Alert behaviour of a class	Admin → Alert rules / <code>alert_rules.json</code>
New audio format	add the extension to <code>SUPPORTED_FORMATS</code> (FFmpeg decodes it)
Maximum upload size / duration	<code>MAX_UPLOAD_MB</code> , <code>MIN_DURATION_S</code> , <code>MAX_DURATION_S</code>
Segment length	Admin → Settings and retrain (the Settings page warns when they differ from the model bundle)
New history filter	<code>filtered_events()</code> in <code>src/routes/main.py</code> + form in <code>templates/history.html</code>
New role	<code>ROLES</code> , <code>@roles_required</code> on routes, navigation in <code>base.html</code> , rule <code>audience</code>
New feature	<code>extract_features()</code> + <code>feature_names()</code> , bump <code>FEATURE_VERSION</code> , retrain
New Python algorithm	add a candidate in <code>train_models.candidates()</code> ; anything with <code>predict_proba</code> works with inference and occlusion-based XAI
New review reason	<code>decide()</code> review-routing block + a test
New admin notification	<code>audit.notify(kind, message)</code> + a check function
PostgreSQL	change <code>SQLALCHEMY_DATABASE_URI</code> in <code>create_app()</code> (JSON columns are stored as text, so no schema change)



CHAPTER 19

Known limitations

What the system cannot do yet and how to improve it.

SRS NFR 4 · Dataset hint

Table 33 Known limitations

Limitation	Impact	Mitigation / next step
GTM accuracy ≈ 0.61 (1-s speech-commands transfer model)	GTM alone misses the 85 % target	<code>python_weight</code> 0.7; disagreements go to review; a larger GTM training set or longer windows are not possible in TM audio projects
Glass Breaking has few clips (116 now, 6 test clips in the evaluated build)	precision 0.56, uncertain recall estimate	add licensed recordings (FSD50K, Freesound CCO) and retrain
Look-alike data gaps (vehicle backfire, metal impact)	those pairs cannot be measured	<code>import_local_downloads.py --sources lookalikes</code> + Freesound clips in <code>downloads/extra/Background Noise/</code>
GTM runs in the browser	uploads need an open tab for GTM; without it the result is Python-only and "Uncertain Result"	server-side TF.js would remove this but would break the SRS independence demonstration
Help detection = fixed safety phrases	not general speech recognition	documented; add phrases to the training data
SQLite	one server process for writes	PostgreSQL for multi-instance deployments
Strong white noise (≤ 10 dB SNR)	accuracy 0.55 – 0.71	more noise augmentation; noise-robust features



CHAPTER 20

Developer troubleshooting

Symptoms, causes and fixes for common developer problems.

SRS SRS 1.8 deliberate defect

Table 34 Developer troubleshooting

Symptom	Cause and fix
ModelNotAvailable: No trained Python model found	run <code>python -m python_models.train_models</code> or copy <code>sonic_model.joblib</code> into <code>python_models/saved/</code>
MP3 upload: "FFmpeg is not installed"	install FFmpeg and restart the terminal so PATH is refreshed
400 "Security token missing or expired"	the session changed (restart with a new secret key); reload the page. fetch calls must use <code>SS.api()</code> so the <code>X-CSRFToken</code> header is sent
First request after start is slow	librosa / numba JIT - <code>warm_up()</code> handles it; it is skipped when <code>WARM_UP=False</code> or in tests
GTM "missing classes" warning	label spelling in <code>metadata.json</code> differs from <code>CLASSES</code>
GTM scores all similar / wrong	check the TM export timestamp; an under-trained export scores $\approx 20\%$ on its own training data (see <code>GTM_TRAINING_LOG.md</code>)
Tests touch real data	never import <code>src</code> before <code>conftest.py</code> sets the environment variables; run tests from the project root
database is locked	long-running script and server writing to SQLite at the same time; stop one or use a separate <code>SONIC_DB_PATH</code>
Changes to templates / CSS not visible	restart the server and hard-refresh (Ctrl + F5)
Teachable Machine rejects zips / crashes while training	paste <code>gtm_model/tm_upload_fix.js</code> into the browser console first



CHAPTER 21

Glossary

Plain definitions of the terms used in this guide.

Table 35 Glossary

Term	Meaning
Audio ID	unique identifier of an analysed recording (<code>AUD-YYYYMMDD-HHMMSS-XXXX</code> ; dataset clips <code>AUD-000001</code>)
Segment / window	fixed-length piece of audio classified by the models (2 s; hop 1 s for uploads)
Clip-level aggregation	turning segment scores into one score vector per recording (<code>aggregate_scores</code>)
Combined score	$\text{python_weight} \cdot \text{Python} + (1 - \text{python_weight}) \cdot \text{GTM}$
Consistency status	Acceptable Match, Weak Match, Model Disagreement or Uncertain Result
$ \Delta\text{conf} $	absolute difference of the two models' top-class confidences
Top-2 margin	difference between the best and second-best class score
Look-alike	a harmless sound that resembles a critical one (fireworks vs gunshot); see <code>LOOKALIKE_PAIRS</code>
Repeated detection	number of consecutive segments / live windows with the same class
Strong confidence	rule field: if both models agree at or above it, one detection confirms the event
Macro-F1	mean F1 over all classes – every class counts equally, regardless of size
Critical recall	recall of Gunshot, Glass Breaking, Panic Scream, Aggression and Help (<code>CRITICAL_RECALL_CLASSES</code>)
GroupKFold	cross-validation that keeps all segments / augmented copies of one recording in the same fold
TreeSHAP	exact feature-contribution method for tree models, used for the XGBoost explanation
<code>BROWSER_FFT</code>	the speech-commands model type exported by Teachable Machine audio projects
SNR	signal-to-noise ratio in dB; lower = noisier
RT60	reverberation time – how long an echo takes to fade by 60 dB
CSRF	cross-site request forgery; blocked with a per-session token on every POST
TMv2	Teachable Machine export format version stored in <code>metadata.json</code>



APPENDIX

Appendix A. Command reference

```
Command reference POWERSHELL / SHELL

1 # environment
2 python -m venv .venv ; .venv\Scripts\activate ; pip install -r requirements.txt
3 python -m database.init_db
4
5 # run
6 python run.py [--host 0.0.0.0] [--port 5000] [--debug]
7 run_server.bat # waitress + auto-restart
8 waitress-serve --port 5000 run:app # Windows production
9 gunicorn -w 1 --threads 4 -b 0.0.0.0:$PORT run:app # Linux production
10
11 # data
12 python scripts/build_dataset.py
13 python -m augmentation.augment --per-clip 2
14
15 # models
16 python -m python_models.train_models [--fast] [--models xgb,mlp,rf]
17 python -m gtm_model.prepare_gtm_samples --max-per-class 400 --zip
18
19 # quality
20 python -m pytest -q
21 python scripts/benchmark_performance.py
22 python scripts/hidden_test_robustness.py
23 python scripts/lookalike_report.py
```



APPENDIX

Appendix B. Related documents

Table 36 Appendix B. Related documents

Document	Content
README.md	overview, installation, usage, surprise modifications
documentation/ARCHITECTURE.md	architecture notes
documentation/DATASET_GUIDE.md, DATA_DICTIONARY.md	data sources, licences, columns
documentation/TESTING.md	test plan and hidden-test checklist
documentation/GTM_TRAINING_LOG.md	Teachable Machine runs and evidence
documentation/DEPLOYMENT.md	Render / Docker deployment
documentation/PROJECT_REPORT.md, TECHNICAL_BLOG.md	report with diagrams, blog
documentation/SRS_COMPLIANCE.md	requirement-by-requirement compliance
documentation/VIVA_PREP.md	module owners, key functions, debugging checklist
AI_USAGE.md, DEVELOPMENT_LOG.md	AI declaration, development log
documentation/ SonicSentinel_AI_User_Guide.docx	end-user guide

Appendix C. Index

A

Aggregation (clip level).....	19
Alert rules.....	10
Alerts – routes.....	28
API reference.....	32
Application factory.....	27
Architecture.....	3
Audit trail.....	34
Augmentation.....	12

B

Benchmark.....	36
----------------	----

C

Configuration.....	9
CSRF.....	27

D

Data model.....	25
Dataset build.....	38
Decision engine.....	22
Deployment – Docker.....	40
Deployment – Render.....	40
Deployment – Windows.....	40

E

Environment variables.....	7
Event life cycle.....	26
Event page anatomy.....	29
Explainable AI (TreeSHAP).....	34
Extending – new class.....	41

F

Feature extraction.....	14
Fingerprint / near-duplicates.....	14

G

GTM – model format.....	20
GTM – results.....	21
GTM – spectrogram.....	20
GTM – training data.....	20

H

Hidden-test robustness.....	36
-----------------------------	----

K

Known limitations.....	42
------------------------	----

L

Live sequence.....	5
Loading audio.....	11

Look-alike check.....	22
M	
Model bundle.....	19
Model independence.....	5
P	
Pre-processing.....	12
Python model – results.....	17
Python model – training.....	16
Q	
Quality analysis.....	11
R	
Repository structure.....	6
Routes.....	28
Runtime settings.....	9
S	
Security.....	27
T	
Test suite.....	35
Troubleshooting.....	43
U	
Upload sequence.....	4
V	
Validation.....	11
W	
Writing tests.....	36